

Vietnam–Korea University of Information and Communication Technology

VKU_TLE_WA_CE

Vo Nguyen Hoang An, Nguyen Tung Lam, Nguyen Van Phung

The 2026 ICPC Vietnam National Contest

Nov 08, 2026

1 Contest 1

2 Toán 1

3 Cấu trúc dữ liệu 4

4 Numerical 6

5 Number theory 8

6 Tổ hợp 11

7 Graph 12

8 Geometry 18

9 Strings 19

10 Various 20

Contest (1)

template.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;
#pragma GCC optimize("O3")
#pragma GCC optimize("unroll-loops")
#define rep(i, a, b) for (int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()
#define all(x) begin(x), end(x)
#define rall(x) rbegin(x), rend(x)
#define nl "\n"
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);
}
```

template.py

```python
import sys
sys.setrecursionlimit(10000000)
from collections import deque, Counter, defaultdict
from bisect import bisect_left, bisect_right, insort
from heapq import (
    heapify, heappush, heappop,
    heappushpop, heapreplace,
    nlargest, nsmallest
)
from itertools import (
    accumulate, permutations, combinations,
    combinations_with_replacement,
    product, groupby, chain
)
from functools import lru_cache, cmp_to_key, reduce
from operator import itemgetter, mul, xor
from fractions import Fraction
from decimal import Decimal, getcontext
getcontext().prec = 20
from array import array
```

.bashrc

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 -
    fsanitize=undefined,address'
alias co='g++ -O2 -std=c++17'
ulimit -s unlimited
xmodmap -e 'clear lock' -e 'keycode 66 = Control_L' -e 'add
    Control = Control_L'
setxkbmap -option ctrl:nocaps
gsettings set org.gnome.desktop.input-sources xkb-options "['
    ctrl:nocaps']"
```

.vimrc

```
set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul
sy on | im jk <esc> | im kj <esc> | no ; :
" Select region and then type :Hash to hash your selection.
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \|
    md5sum \| cut -c-6
nnoremap <F1> :w<CR>:!g++ -Wall -Wconversion -Wfatal-errors -g
    -std=c++17 -fsanitize=undefined,address % -o %:r && ./%:r<
    CR>
```

hash.sh

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

troubleshoot.txt

```
- Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.
- Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.
- Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).
- Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?
- Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
```

Toán (2)

2.1 Phương trình

2.1.1 Phương trình bậc hai

Xét phương trình $ax^2 + bx + c = 0$ ($a \neq 0$), $\Delta = b^2 - 4ac$:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Tọa độ đỉnh parabol (cực trị): Đạt tại $x = -\frac{b}{2a}$, giá trị tương ứng $y = -\frac{\Delta}{4a}$

Định lý Viète: $x_1 + x_2 = \frac{b}{a} \quad x_1 x_2 = \frac{c}{a}$

2.1.2 Hệ phương trình bậc nhất hai ẩn (Quy tắc Cramer 2x2)

Dùng để giải hệ phương trình tuyến tính hoặc tìm nhanh giao điểm 2 đường thẳng:

$$\begin{cases} ax + by = e \\ cx + dy = f \end{cases} \implies x = \frac{ed - bf}{ad - bc}, \quad y = \frac{af - ec}{ad - bc}$$

Nếu định thức $D = ad - bc = 0$: Hệ vô nghiệm (hai đường song song) hoặc vô số nghiệm (hai đường trùng nhau).

2.1.3 Hệ phương trình tuyến tính tổng quát

Cho hệ $Ax = b$ với A là ma trận vuông không suy biến ($\det A \neq 0$):

$$x_i = \frac{\det A'_i}{\det A}$$

Trong đó A'_i là ma trận nhận được từ ma trận A sau khi thay thế cột thứ i bằng vector cột b. (Thực tế chỉ dùng nhẩm tay cho hệ 2×2 và 3×3; hệ lớn hơn giải bằng khử Gauss).

2.2 Công thức truy hồi (Recurrences)

2.2.1 Phương trình đặc trưng (Hệ thức truy hồi tuyến tính thuần nhất)

Xét hệ thức truy hồi tuyến tính cấp k:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k}$$

Thiết lập phương trình đa thức đặc trưng:

$$P(x) = x^k - c_1 x^{k-1} - c_2 x^{k-2} - \cdots - c_k = 0$$

Trường hợp nghiệm phân biệt: Nếu phương trình có k nghiệm phân biệt $r_1, r_2, \ldots, r_k$, công thức tổng quát là:

$$a_n = d_1 r_1^n + d_2 r_2^n + \cdots + d_k r_k^n$$

(Xác định các hệ số d_i bằng cách thế các giá trị khởi tạo ban đầu $a_0, a_1, \ldots, a_{k-1}$ để giải hệ phương trình tuyến tính)
Trường hợp nghiệm bội: Nếu nghiệm r lặp lại m lần (bội m), nó sẽ đóng góp một nhân tử đa thức bậc $m-1$:

$$(d_1 + d_2 n + d_3 n^2 + \cdots + d_m n^{m-1}) r^n$$

Ví dụ dãy Fibonacci: $F_n = F_{n-1} + F_{n-2}$ với $F_0 = 0, F_1 = 1$.
Phương trình đặc trưng: $x^2 - x - 1 = 0 \implies r = \frac{1 \pm \sqrt{5}}{2}$.
Công thức Binet dạng đóng:

$$F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right]$$

2.2.2 Nhân ma trận giải công thức truy hồi (Matrix Exponentiation)

Áp dụng khi cần tính số hạng thứ n với giá trị n rất lớn ($n \le 10^{18}$) và bậc truy hồi nhỏ ($k \le 50$) trong độ phức tạp $\mathcal{O}(k^3 \log n)$:

$$\begin{bmatrix} a_n \\ a_{n-1} \\ a_{n-2} \\ \vdots \\ a_{n-k+1} \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 & \ldots & c_k \\ 1 & 0 & 0 & \ldots & 0 \\ 0 & 1 & 0 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & 0 \end{bmatrix} \begin{bmatrix} a_{n-1} \\ a_{n-2} \\ a_{n-3} \\ \vdots \\ a_{n-k} \end{bmatrix}$$

2.3 Lượng giác

$$a\cos x + b\sin x = r\cos(x - \phi)$$
$$a\sin x + b\cos x = r\sin(x + \phi)$$

với $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Hình học

2.4.1 Tam giác

Cho tam giác có độ dài ba cạnh a, b, c. **Nửa chu vi:**

$$p = \frac{a+b+c}{2}$$

Diện tích (công thức Heron):

$$A = \sqrt{p(p-a)(p-b)(p-c)}$$

Bán kính đường tròn ngoại tiếp:

$$R = \frac{abc}{4A}$$

Bán kính đường tròn nội tiếp:

$$r = \frac{A}{p}$$

Độ dài đường trung tuyến: Đường trung tuyến chia tam giác thành hai tam giác có diện tích bằng nhau.

$$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$$

Độ dài đường phân giác: Đường phân giác chia một góc thành hai góc bằng nhau.

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Định lý sin:

$$\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$$

Định lý cosin:

$$a^2 = b^2 + c^2 - 2bc\cos\alpha$$

Định lý tang:

$$\frac{a+b}{a-b} = \frac{\tan\left(\frac{\alpha+\beta}{2}\right)}{\tan\left(\frac{\alpha-\beta}{2}\right)}$$

2.4.2 Tứ giác

Cho tứ giác có độ dài các cạnh a, b, c, d, hai đường chéo e, f, góc giữa hai đường chéo θ, diện tích A và đặt

$$F = b^2 + d^2 - a^2 - c^2.$$

Ta có:

$$4A = 2ef\sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}.$$

Đối với tứ giác nội tiếp, tổng hai góc đối bằng 180°. Ngoài ra, ta có:

$$ef = ac + bd,$$
$$A = \sqrt{(p-a)(p-b)(p-c)(p-d)},$$

trong đó $p = \frac{a+b+c+d}{2}$ là nửa chu vi tứ giác.

2.5 Tổng

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, \quad c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$
$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$
$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.5.1 Chia kẹo (Stars and Bars)

Số nghiệm nguyên của $x_1 + x_2 + \cdots + x_k = n$ (chia n vật giống nhau vào k hộp phân biệt): $x_i \ge 0$: $\binom{n+k-1}{k-1}$
$x_i \ge 1$ (cần $n \ge k$): $\binom{n-1}{k-1}$
$x_i \ge c_i$: đặt $n' = n - \sum c_i$, nếu $n' \ge 0$ thì đáp án là $\binom{n'+k-1}{k-1}$, ngược lại bằng 0
$x_i \le u_i$: bù trừ theo tập S các biến vi phạm ($x_i \ge u_i + 1$): $\sum_{S \subseteq [k]} (-1)^{|S|} \binom{n - \sum_{i \in S}(u_i+1) + k - 1}{k-1}$
bỏ các số hạng có $n - \sum_{i \in S}(u_i + 1) < 0$. Nếu mọi $u_i = u$ thì $\sum_{j=0}^{k} (-1)^j \binom{k}{j} \binom{n - j(u+1) + k - 1}{k-1}$

2.6 Hệ số nhị thức và đa thức

2.6.1 Công thức cơ bản

$$\binom{n}{r} = \frac{n!}{r!(n-r)!} = \binom{n}{n-r}, \binom{n}{r} = 0 \quad (r < 0 \text{ hoặc } r > n)$$
$$\binom{n}{r} = \binom{n-1}{r-1} + \binom{n-1}{r} = \binom{n}{r-1}\frac{n-r+1}{r}.$$
$$\sum_r \binom{n}{r} = 2^n, \quad \sum_r \binom{n}{r}^2 = \binom{2n}{n}.$$
$$\sum_{i=0}^{r} \binom{m}{i}\binom{n}{r-i} = \binom{m+n}{r}, \sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}.$$
$\binom{n}{r}$ lẻ $\iff$ `(n & r) == r`.

2.6.2 Tính $\binom{n}{r}$

BinomExact.h

Description: hàm này để tính chính xác $\binom{n}{r}$ (không lấy modulo) bằng cách nhân chia xen kẽ. Sau bước i, `res` $= \binom{n}{i+1}$ nên phép chia luôn chia hết. Kết quả phải vừa trong `ll`; tích trung gian dùng `__int128` để không bị tràn. Trả về 0 nếu $r < 0$ hoặc $r > n$.
Time: $\mathcal{O}(\min(r, n-r))$

14a3ef, 9 lines

```
ll Cexact(ll n, ll r) {
    if (r < 0 || r > n)
        return 0;
    r = min(r, n - r);
    ll res = 1;
    for (ll i = 0; i < r; i++)
```

```
        res = (__int128)res * (n - i) / (i + 1);
    return res;
}
```

BinomMod.h

Description: hàm này để tính $\binom{n}{r}$ mod p với p nguyên tố và $n \le \min(N-1, p-1)$. `init()` tiền xử lý giai thừa và nghịch đảo giai thừa (đặt `MOD` trước khi gọi), sau đó mỗi truy vấn `C(n, r)` mất $\mathcal{O}(1)$. Nếu $n \ge p$ thì dùng `lucas` trong Lucas.h.
Usage: `MOD = 1000000007; init(); C(10, 3);`
Time: $\mathcal{O}(N)$, mỗi truy vấn $\mathcal{O}(1)$

bc7d44, 24 lines

```
const int N = 1000005;
ll MOD = 1000000007, fact[N], ifact[N];
ll pw(ll a, ll e) {
    ll r = 1;
    a %= MOD;
    for (; e; e >>= 1, a = a * a % MOD)
        if (e & 1)
            r = r * a % MOD;
    return r;
}
void init() {
    int L = min<ll>(N - 1, MOD - 1);
    fact[0] = 1;
    for (int i = 1; i <= L; i++)
        fact[i] = fact[i - 1] * i % MOD;
    ifact[L] = pw(fact[L], MOD - 2);
    for (int i = L; i; i--)
        ifact[i - 1] = ifact[i] * i % MOD;
}
ll C(ll n, ll r) {
    if (r < 0 || r > n)
        return 0;
    return fact[n] * ifact[r] % MOD * ifact[n - r] % MOD;
}
```

2.6.3 Định lý Lucas

Cho p là số nguyên tố. Viết biểu diễn cơ số p của n và r:

$$n = \sum_{i=0}^{k} n_i p^i, \qquad r = \sum_{i=0}^{k} r_i p^i.$$

Theo định lý Lucas:

$$\binom{n}{r} \equiv \prod_{i=0}^{k} \binom{n_i}{r_i} \pmod{p}.$$

Kết quả bằng 0 nếu tồn tại i sao cho $n_i < r_i$.

Lucas.h

Description: hàm này để tính $\binom{n}{r}$ mod p với p là số nguyên tố nhỏ ($p \le N$) còn n, r rất lớn (tới 10^{18}). Theo định lý Lucas, $\binom{n}{r} \equiv \prod_i \binom{n_i}{r_i} \pmod{p}$ với n_i, r_i là các chữ số của n, r trong hệ cơ số p (bằng 0 nếu có $n_i < r_i$). Cần BinomMod.h: đặt `MOD` $= p$ rồi gọi `init()` trước.
Usage: `MOD = 99991; init(); lucas(1e18, 12345);`
Time: $\mathcal{O}(p + \log_p n)$

d48bc5, 8 lines

```
ll lucas(ll n, ll r) {
    if (r < 0 || r > n)
        return 0;
    ll res = 1;
    for (; n || r; n /= MOD, r /= MOD)
        res = res * C(n % MOD, r % MOD) % MOD;
    return res;
}
```

2.6.4 Tổng $\sum_{k=0}^{n} P(k)\binom{n}{k}$ với n lớn, P bậc nhỏ

Đặt giai thừa giảm:

$$k^{\underline{r}} = k(k-1)\cdots(k-r+1).$$

Ta có đồng nhất thức:

$$k^{\underline{r}}\binom{n}{k} = n^{\underline{r}}\binom{n-r}{k-r}.$$

Suy ra:

$$\sum_{k=0}^{n} k^{\underline{r}}\binom{n}{k} = n^{\underline{r}}\sum_{k=r}^{n}\binom{n-r}{k-r} = n^{\underline{r}}2^{n-r}.$$

Quy ước kết quả bằng 0 nếu $r > n$.

Một số trường hợp thường gặp:

$$\sum_{k=0}^{n} k\binom{n}{k} = n2^{n-1}$$

$$\sum_{k=0}^{n} k^2\binom{n}{k} = n(n+1)2^{n-2}.$$

SumPoly.h

Description: hàm này để tính $\sum_{k=0}^{n} P(k)\binom{n}{k}$ theo modulo MOD, với n rất lớn (tới 10^{18}) và P có bậc nhỏ. Viết $P(k) = \sum_r c_r (k)_r$ với $(k)_r = k(k-1)\cdots(k-r+1)$ là giai thừa giảm, khi đó tổng bằng $\sum_r c_r (n)_r 2^{n-r}$. Đổi cơ sở: $k^2 = (k)_2 + (k)_1$, $k^3 = (k)_3 + 3(k)_2 + (k)_1$, $k^4 = (k)_4 + 6(k)_3 + 7(k)_2 + (k)_1$. Cần BinomMod.h cho pw và MOD.
Usage: `sumPoly(n, {0, 1, 1}); // tong k*k*C(n,k)`
Time: $\mathcal{O}(d \log n)$ với d là bậc của P

1a9318, 8 lines

```
ll sumPoly(ll n, const vector<ll> &c) {
    ll res = 0, f = 1; // f = (n)_r mod MOD
    for (int r = 0; r < (int)c.size() && r <= n; r++) {
        res = (res + c[r] % MOD * f % MOD * pw(2, n - r)) % MOD
            ;
        f = f * ((n - r) % MOD) % MOD;
    }
    return res;
}
```

2.6.5 Hệ số đa thức và xâu thứ k theo từ điển

Số hoán vị của n phần tử, loại i có a_i phần tử:
$\binom{n}{a_1,\ldots,a_m} = \frac{n!}{a_1!\cdots a_m!} = \binom{n}{a_1}\binom{n-a_1}{a_2}\cdots$.

KthPermutation.h

Description: hàm này để tìm xâu (hoán vị có lặp) thứ k theo thứ tự từ điển, k đánh số từ 1. `a[i]` là số lần xuất hiện của ký tự `'a'+i`. Trả về xâu rỗng nếu k lớn hơn tổng số hoán vị. Thuật toán tham lam: ở mỗi vị trí thử ký tự c tăng dần, `cnt` là số hoán vị của phần còn lại $\binom{n-1}{a_1,\ldots,a_c-1,\ldots,a_m}$; nếu $k \le$ `cnt` thì chốt c, ngược lại k `-= cnt`. Mọi phép cộng/nhân bị chặn trên bởi `INF` để không tràn số (không cần BigInt). Gọi `initPascal()` một lần ở đầu. Usage: initPascal(); kth(2, 1, 1, 5); // các ký tự a,a,b,c: aabc, aacb, abac, abca, acab => "acab" Time: $\mathcal{O}(M^2)$ khởi tạo, $\mathcal{O}(n \cdot m^2)$ mỗi truy vấn Status: chưa test

d1e070, 38 lines

```
const ll INF = 2e18;
const int M = 2005; // độ dài xâu tối đa
ll Cs[M][M];
void initPascal() {
    for (int i = 0; i < M; i++) {
        Cs[i][0] = 1;
        for (int j = 1; j <= i; j++)
            Cs[i][j] = min(INF, Cs[i - 1][j - 1] + Cs[i - 1][j
                ]);
    }
}
ll mulSat(ll a, ll b) { return (a && b > INF / a) ? INF : a * b
    ; }
ll multinomial(const vector<int> &a, int n) {
    ll res = 1;
    for (int x : a) {
        res = mulSat(res, Cs[n][x]);
        n -= x;
    }
    return res;
}
string kth(vector<int> a, ll k) {
    int n = accumulate(a.begin(), a.end(), 0);
    if (multinomial(a, n) < k)
        return "";
    string s;
    for (int pos = 0; pos < n; pos++)
        for (int c = 0; c < (int)a.size(); c++)
            if (a[c]) {
                a[c]--;
                ll cnt = multinomial(a, n - pos - 1);
                if (k <= cnt) {
                    s += char('a' + c);
                    break;
                }
                k -= cnt;
                a[c]++;
            }
    return s;
}
```

2.7 Lý thuyết xác suất

Gọi X là biến ngẫu nhiên rời rạc với xác suất nhận giá trị x là $p_X(x)$. Khi đó, giá trị kỳ vọng (trung bình) của nó là $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ và phương sai là $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$, trong đó σ là độ lệch chuẩn. Nếu X là biến ngẫu nhiên liên tục, nó sẽ có hàm mật độ xác suất $f_X(x)$, và các tổng ở trên được thay bằng tích phân với $p_X(x)$ thay bằng $f_X(x)$
Kỳ vọng có tính tuyến tính:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

Đối với X và Y độc lập:

$$V(aX + bY) = a^2 V(X) + b^2 V(Y)$$

2.7.1 Các phân phối rời rạc

Phân phối nhị thức (Binomial distribution)

Số lần thành công trong n phép thử độc lập có kết quả có/không (thành công/thất bại), mỗi phép thử có xác suất thành công là p, tuân theo phân phối $\text{Bin}(n,p)$, $n = 1, 2, \ldots$, $0 \le p \le 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\text{Bin}(n,p)$ xấp xỉ $\text{Po}(np)$ (phân phối Poisson) khi p nhỏ.

Phân phối số lần thử tới thành công đầu tiên (First success distribution)

Số phép thử cần thiết để nhận được lần thành công đầu tiên trong các phép thử độc lập có kết quả có/không, mỗi phép thử có xác suất thành công là p, tuân theo phân phối $\text{Fs}(p)$, $0 \le p \le 1$ (dạng hình học dịch chuyển/Geometric distribution bắt đầu từ 1).

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

Cấu trúc dữ liệu (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change `null_type`.
Time: $\mathcal{O}(\log N)$

782797, 16 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

void example() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).first;
  assert(it == t.lower_bound(9));
  assert(t.order_of_key(10) == 1);
  assert(t.order_of_key(11) == 2);
  assert(*t.find_by_order(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d77092, 7 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

SegmentTree.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.
Time: $\mathcal{O}(\log N)$

0f4bdb, 19 lines

```
struct Tree {
  typedef int T;
  static constexpr T unit = INT_MIN;
  T f(T a, T b) { return max(a, b); } // (any associative fn)
  vector<T> s; int n;
  Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
  void update(int pos, T val) {
    for (s[pos += n] = val; pos /= 2;)
      s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
  }
  T query(int b, int e) { // query [b, e)
    T ra = unit, rb = unit;
    for (b += n, e += n; b < e; b /= 2, e /= 2) {
      if (b % 2) ra = f(ra, s[b++]);
      if (e % 2) rb = f(s[--e], rb);
    }
    return f(ra, rb);
  }
};
```

LazySegmentTree.h

Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.
Usage: `Node* tr = new Node(v, 0, sz(v));`
Time: $\mathcal{O}(\log N)$.

"../various/BumpAllocator.h" 34ecf5, 50 lines

```
const int inf = 1e9;
struct Node {
  Node *l = 0, *r = 0;
  int lo, hi, mset = inf, madd = 0, val = -inf;
  Node(int lo,int hi):lo(lo),hi(hi){} // Large interval of -inf
  Node(vi& v, int lo, int hi) : lo(lo), hi(hi) {
    if (lo + 1 < hi) {
      int mid = lo + (hi - lo)/2;
      l = new Node(v, lo, mid); r = new Node(v, mid, hi);
      val = max(l->val, r->val);
    }
    else val = v[lo];
  }
  int query(int L, int R) {
    if (R <= lo || hi <= L) return -inf;
    if (L <= lo && hi <= R) return val;
    push();
    return max(l->query(L, R), r->query(L, R));
  }
  void set(int L, int R, int x) {
    if (R <= lo || hi <= L) return;
    if (L <= lo && hi <= R) mset = val = x, madd = 0;
    else {
      push(), l->set(L, R, x), r->set(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void add(int L, int R, int x) {
    if (R <= lo || hi <= L) return;
    if (L <= lo && hi <= R) {
      if (mset != inf) mset += x;
      else madd += x;
      val += x;
    }
    else {
      push(), l->add(L, R, x), r->add(L, R, x);
      val = max(l->val, r->val);
    }
  }
  void push() {
    if (!l) {
      int mid = lo + (hi - lo)/2;
      l = new Node(lo, mid); r = new Node(mid, hi);
    }
    if (mset != inf)
      l->set(lo,hi,mset), r->set(lo,hi,mset), mset = inf;
    else if (madd)
      l->add(lo,hi,madd), r->add(lo,hi,madd), madd = 0;
  }
};
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

de4ad0, 21 lines

```
struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

SubMatrix.h

Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).
Usage: `SubMatrix<int> m(matrix);`
`m.sum(0, 0, 2, 2); // top left 4 elements`
Time: $\mathcal{O}\left(N^2 + Q\right)$

c59ada, 13 lines

```
template<class T>
struct SubMatrix {
  vector<vector<T>> p;
  SubMatrix(vector<vector<T>>& v) {
    int R = sz(v), C = sz(v[0]);
    p.assign(R+1, vector<T>(C+1));
    rep(r,0,R) rep(c,0,C)
      p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
  }
  T sum(int u, int l, int d, int r) {
    return p[d][r] - p[d][l] - p[u][r] + p[u][l];
  }
};
```

Matrix.h

Description: Basic operations on square matrices.
Usage:
```
Matrix<int, 3> A;
A.d = {{{{1,2,3}}, {{4,5,6}}, {{7,8,9}}}};
array<int, 3> vec = {1,2,3};
vec = (A^N) * vec;
```
<cassert> 4da5a2, 26 lines

```
template<class T, int N> struct Matrix {
  typedef Matrix M;
  array<array<T, N>, N> d{};
  M operator*(const M& m) const {
    M a;
    rep(i,0,N) rep(j,0,N)
      rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
    return a;
  }
  array<T, N> operator*(const array<T, N>& vec) const {
    array<T, N> ret{};
    rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
    return ret;
  }
  M operator^(ll p) const {
    assert(p >= 0);
    M a, b(*this);
    rep(i,0,N) a.d[i][i] = 1;
    while (p) {
      if (p&1) a = a*b;
      b = b*b;
      p >>= 1;
    }
    return a;
  }
};
```

LineContainer.h

Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").
Time: $\mathcal{O}(\log N)$
<cassert> 8ec1c7, 30 lines

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

FenwickTree.h

Description: Computes partial sums a[0] + a[1] + ... + a[pos - 1], and updates single elements a[i], taking the difference between the old and new value.
Time: Both operations are $\mathcal{O}(\log N)$.
e62fac, 22 lines

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
  }
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos > 0; pos &= pos - 1) res += s[pos-1];
    return res;
  }
  int lower_bound(ll sum) {// min pos st sum of [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
        pos += pw, sum -= s[pos-1];
    }
    return pos;
  }
};
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all i<I, j<J, and increases single elements a[i,j]. Requires that the elements to be updated are known in advance (call fakeUpdate() before init()).
Time: $\mathcal{O}\left(\log^2 N\right)$. (Use persistent segment trees for $\mathcal{O}(\log N)$.)
"FenwickTree.h" 157f07, 22 lines

```
struct FT2 {
  vector<vi> ys; vector<FT> ft;
  FT2(int limx) : ys(limx) {}
  void fakeUpdate(int x, int y) {
    for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
  }
  void init() {
    for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
  }
  int ind(int x, int y) {
    return (int)(lower_bound(all(ys[x]), y) - ys[x].begin()); }
  void update(int x, int y, ll dif) {
    for (; x < sz(ys); x |= x + 1)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x; x &= x - 1)
      sum += ft[x-1].query(ind(x-1, y));
    return sum;
  }
};
```

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage:
```
RMQ rmq(values);
rmq.query(inclusive, exclusive);
```
Time: $\mathcal{O}(|V| \log |V| + Q)$
<cassert> 510c32, 16 lines

```
template<class T>
struct RMQ {
  vector<vector<T>> jmp;
  RMQ(const vector<T>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw *= 2, ++k) {
      jmp.emplace_back(sz(V) - pw * 2 + 1);
      rep(j,0,sz(jmp[k]))
        jmp[k][j] = min(jmp[k - 1][j], jmp[k - 1][j + pw]);
    }
  }
  T query(int a, int b) {
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 << dep)]);
  }
};
```

BinaryTrie.h

Description: Binary Trie (0-1 Trie) for dynamic insertion, deletion, and bitwise XOR queries (maximum, minimum, and count <= k) on non-negative integers.
Usage:
```
BinaryTrie tr;
tr.insert(x); tr.erase(x);
tr.maxXor(x); // returns max (x ^ y) for y in trie
tr.minXor(x); // returns min (x ^ y) for y in trie
tr.countLeq(x, k); // count y in trie such that (x ^ y) <= k
```
Time: $\mathcal{O}(B)$ per operation, where B is the bit length (B = 30 for <= 1e9).
<cassert> 35c8cd, 68 lines

```
struct BinaryTrie {
  static const int B = 30; // 30 bits for values < 2^30 (~1e9)
  struct Node {
    int ch[2] = {0, 0};
    int cnt = 0;
  };
  vector<Node> t = {Node()};
  void insert(int x, int v = 1) { // v = -1 to erase
    int u = 0;
    t[u].cnt += v;
    for (int i = B - 1; i >= 0; --i) {
      int b = (x >> i) & 1;
      if (!t[u].ch[b]) {
        t[u].ch[b] = sz(t);
        t.emplace_back();
      }
      u = t[u].ch[b];
      t[u].cnt += v;
    }
  }
  void erase(int x) { insert(x, -1); }
  int maxXor(int x) {
    assert(t[0].cnt > 0);
    int u = 0, res = 0;
    for (int i = B - 1; i >= 0; --i) {
      int b = (x >> i) & 1;
      if (t[u].ch[!b] && t[t[u].ch[!b]].cnt > 0) {
        res |= (1 << i);
        u = t[u].ch[!b];
      } else {
        u = t[u].ch[b];
      }
    }
    return res;
  }
  int minXor(int x) {
    assert(t[0].cnt > 0);
    int u = 0, res = 0;
    for (int i = B - 1; i >= 0; --i) {
      int b = (x >> i) & 1;
      if (t[u].ch[b] && t[t[u].ch[b]].cnt > 0) {
```

```
      u = t[u].ch[b];
    } else {
      res |= (1 << i);
      u = t[u].ch[!b];
    }
  }
  return res;
}
// Count number of elements y in trie such that (x ^ y) <= k
int countLeq(int x, int k) {
      if (k < 0) return 0;
      if (k >= (1 << B)) return t[0].cnt;

      int u = 0, ans = 0;
      for (int i = B - 1; i >= 0; --i) {
          int bx = (x >> i) & 1, bk = (k >> i) & 1;
          if (bk) {
              if (t[u].ch[bx]) ans += t[t[u].ch[bx]].cnt;
              u = t[u].ch[!bx];
          } else {
              u = t[u].ch[bx];
          }
          if (!u) return ans;
      }
      return ans + t[u].cnt;
  }
};
```

MoQueries.h

Description: Answer interval or tree path queries by finding an approximate TSP through the queries, and moving from one query to the next by adding/removing points at the ends. If values are on tree edges, change `step` to add/remove the edge (a, c) and remove the initial `add` call (but keep `in`).
Time: $\mathcal{O}\left(N\sqrt{Q}\right)$

a12ef4, 49 lines

```
void add(int ind, int end) { ... } // add a[ind] (end = 0 or 1)
void del(int ind, int end) { ... } // remove a[ind]
int calc() { ... } // compute current answer

vi mo(vector<pii> Q) {
  int L = 0, R = 0, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;
#define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) {
    pii q = Q[qi];
    while (L > q.first) add(--L, 0);
    while (R < q.second) add(R++, 1);
    while (L < q.first) del(L++, 0);
    while (R > q.second) del(--R, 1);
    res[qi] = calc();
  }
  return res;
}

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(0, 0), in[0] = 1;
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  };
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int s, int t){ return K(Q[s]) < K(Q[t]); });
  for (int qi : s) rep(end,0,2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; }
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  }
  return res;
}
```

Numerical (4)

4.1 Optimization

GoldenSectionSearch.h

Description: Tìm vị trí cực tiểu của hàm f trên đoạn [a, b], với giả thiết f là hàm đơn cực (unimodal) trên đoạn này, tức là chỉ có một điểm cực tiểu địa phương và không có điểm cực đại địa phương. Sai số tối đa của kết quả là eps. Có thể sử dụng tương tự để tìm cực đại bằng cách thay đổi một điều kiện nhỏ trong code. Xem TernarySearch.h trong chương Various để biết phiên bản tìm kiếm trên miền rời rạc.
Usage: `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func);`
Time: $\mathcal{O}\left(log((b-a)/eps)\right)$

31d45b, 20 lines

```
double gss(double a, double b, double (*f)(double)) {
  double r = (sqrt(5) - 1) / 2, eps = 1e-7;
  double x1 = b - r * (b - a), x2 = a + r * (b - a);
  double f1 = f(x1), f2 = f(x2);
  while (b - a > eps)
    if (f1 < f2) { // đổi thành > nếu muốn tìm cực đại
      b = x2;
      x2 = x1;
      f2 = f1;
      x1 = b - r * (b - a);
      f1 = f(x1);
    } else {
      a = x1;
      x1 = x2;
      f1 = f2;
      x2 = a + r * (b - a);
      f2 = f(x2);
    }
  return a;
}
```

4.2 Matrices

IntDeterminant.h

Description: Tính định thức của ma trận bằng các phép toán số học modulo. Có thể bỏ phép lấy modulo để thu được phiên bản tính định thức trên số nguyên thông thường.
Time: $\mathcal{O}\left(N^3\right)$

3313dc, 20 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>> &a) {
  int n = sz(a);
  ll ans = 1;
  rep(i, 0, n) {
    rep(j, i + 1, n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t)
          rep(k, i, n) a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans)
      return 0;
  }
  return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Giải hệ phương trình tuyến tính A * x = b bằng phương pháp khử Gauss. Nếu hệ có nhiều nghiệm, trả về một nghiệm bất kỳ. Trả về hạng (rank) của ma trận A, hoặc -1 nếu hệ vô nghiệm. Nếu rank < m (số ẩn), hệ có vô số nghiệm. Dữ liệu trong A và b sẽ bị thay đổi trong quá trình thực hiện.
Time: $\mathcal{O}\left(n^2 * m\right)$

<cassert> 44c9ab, 36 lines

```
typedef vector<double> vd;
const double eps = 1e-12;
int solveLinear(vector<vd> &A, vd &b, vd &x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n)
    assert(sz(A[0]) == m);
  vi col(m);
  iota(all(col), 0);
  rep(i, 0, n) {
    double v, bv = 0;
    rep(r, i, n) rep(c, i, m) if ((v = fabs(A[r][c])) > bv) br
        = r, bc = c,
                                  bv = v;
    if (bv <= eps) {
      rep(j, i, n) if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j, 0, n) swap(A[j][i], A[j][bc]);
    bv = 1 / A[i][i];
    rep(j, i + 1, n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k, i + 1, m) A[j][k] -= fac * A[i][k];
    }
    rank++;
  }
  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j, 0, i) b[j] -= A[j][i] * b[i];
  }
  return rank; // rank < m nếu hệ có vô số nghiệm
}
```

SolveLinearBinary.h

Description: Giải hệ phương trình $Ax = b$ trên $\mathbb{F}_2$. Nếu có nhiều nghiệm, trả về một nghiệm bất kỳ. Trả về hạng của ma trận, hoặc -1 nếu hệ vô nghiệm. Phá hủy dữ liệu trong A và b.
Time: $\mathcal{O}\left(n^2 m\right)$

<cassert> fa2d7a, 39 lines

```
typedef bitset<1000> bs;

int solveLinear(vector<bs> &A, vi &b, bs &x, int m) {
  int n = sz(A), rank = 0, br;
```

```
  assert(m <= sz(x));
  vi col(m);
  iota(all(col), 0);
  rep(i, 0, n) {
    for (br = i; br < n; ++br)
      if (A[br].any())
        break;
    if (br == n) {
      rep(j, i, n) if (b[j]) return -1;
      break;
    }
    int bc = (int)A[br]._Find_next(i - 1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j, 0, n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i);
      A[j].flip(bc);
    }
    rep(j, i + 1, n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  }

  x = bs();
  for (int i = rank; i--;) {
    if (!b[i])
      continue;
    x[col[i]] = 1;
    rep(j, 0, i) b[j] ^= A[j][i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

MatrixInverse.h

Description: Tính ma trận nghịch đảo của A. Trả về hạng của ma trận; kết quả được lưu trực tiếp vào A, trừ trường hợp ma trận suy biến (rank < n). Có thể dễ dàng mở rộng cho modulo số nguyên tố. Với modulo lũy thừa của số nguyên tố, lặp lại phép cập nhật: $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ trong đó A^{-1} ban đầu là ma trận nghịch đảo của A modulo p, và k được nhân đôi sau mỗi bước.
Time: $\mathcal{O}(n^3)$

ebffff6, 37 lines

```
int matInv(vector<vector<double>> &A) {
  int n = sz(A);
  vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i, 0, n) tmp[i][i] = 1, col[i] = i;

  rep(i, 0, n) {
    int r = i, c = i;
    rep(j, i, n) rep(k, i, n) if (fabs(A[j][k]) > fabs(A[r][c])
        ) r = j,
                                  c = k;
    if (fabs(A[r][c]) < 1e-12)
      return i;
    A[i].swap(A[r]);
    tmp[i].swap(tmp[r]);
    rep(j, 0, n) swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j
        ][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j, i + 1, n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k, i + 1, n) A[j][k] -= f * A[i][k];
      rep(k, 0, n) tmp[j][k] -= f * tmp[i][k];
    }
    rep(j, i + 1, n) A[i][j] /= v;
    rep(j, 0, n) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  for (int i = n - 1; i > 0; --i)
    rep(j, 0, i) {
      double v = A[j][i];
      rep(k, 0, n) tmp[j][k] -= v * tmp[i][k];
    }

  rep(i, 0, n) rep(j, 0, n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

4.3 Fourier transforms

FastFourierTransform.h

Description: fft(a) tính $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ với mọi k. N phải là lũy thừa của 2. Ứng dụng hữu ích: tính tích chập (convolution): conv(a, b) = c, trong đó $c[x] = \sum a[i]b[x-i]$. Để tính tích chập của các số phức hoặc của nhiều hơn hai vector: thực hiện FFT, nhân từng phần tử tương ứng, chia cho n, đảo ngược đoạn [start+1, end), rồi thực hiện FFT lần nữa. Phép làm tròn đảm bảo chính xác nếu $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (trong thực tế có thể lên đến 10^{16}; cao hơn với dữ liệu ngẫu nhiên). Nếu không thỏa mãn, hãy dùng NTT/FFTMod.
Time: $\mathcal{O}(N \log N)$ với $N = |A| + |B|$ ($\sim 1s$ khi $N = 2^{22}$)

be03a2, 42 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C> &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1); // (^ 10% nhanh hơn nếu dùng
      double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n);
    rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i, k, 2 * k) rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i
        / 2];
  }
  vi rev(n);
  rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k)
      rep(j, 0, k) {
        // C z = rt[j + k] * a[i + j + k];
        (nhanh hơn 25% nếu tự viết phép nhân) include-line
        auto x = (double *)&rt[j + k],
        C z(x[0] * y[0] - x[1] * y[1],
        a[i + j + k] = a[i + j] - z;
        a[i + j] += z;
      }
}
vd conv(const vd &a, const vd &b) {
  if (a.empty() || b.empty())
    return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i, 0, sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C &x : in)
    x *= x;
  rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i, 0, sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
}
```

FastFourierTransformMod.h

Description: FFT với độ chính xác cao hơn, có thể dùng để tính tích chập modulo một số nguyên bất kỳ, miễn là $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (trong thực tế có thể lên đến 10^{16} hoặc cao hơn). Các phần tử đầu vào phải thuộc đoạn $[0, \text{mod})$.
Time: $\mathcal{O}(N \log N)$, với $N = |A| + |B|$ (chậm gấp đôi NTT hoặc FFT thông thường)

"FastFourierTransform.h" b82773, 23 lines

```
typedef vector<ll> vl;
template <int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty())
    return {};
  vl res(sz(a) + sz(b) - 1);
  int B = 32 - __builtin_clz(sz(res)), n = 1 << B, cut = int(
      sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i, 0, sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i, 0, sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  rep(i, 0, n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  rep(i, 0, sz(res)) {
    ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) + .5)
        ;
    ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) tính $\hat{f}(k) = \sum_x a[x] g^{xk}$ với mọi k, trong đó $g = \text{root}^{(mod-1)/N}$. N phải là lũy thừa của 2. Hữu ích để tính tích chập modulo các số nguyên tố đặc biệt có dạng $2^a b + 1$, trong đó kích thước kết quả tích chập không vượt quá 2^a. Với modulo bất kỳ, xem FFTMod. conv(a, b) = c, trong đó $c[x] = \sum a[i]b[x-i]$. Nếu tự tính tích chập: thực hiện NTT trên các đầu vào, nhân từng phần tử tương ứng, chia cho n, đảo ngược đoạn [start+1, end), rồi thực hiện NTT lần nữa. Các phần tử đầu vào phải thuộc $[0, \text{mod})$.
Time: $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" d54056, 35 lines

```
const ll mod = (119 << 23) + 1, root = 62; // = 998244353
// Với p < 2^30 còn có thể dùng, chẳng hạn: 5 << 25, 7 << 26,
    479 << 21
// và 483 << 21 (hai giá trị cuối lớn hơn 10^9).
typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};
    rep(i, k, 2 * k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  }
  vi rev(n);
  rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i, 0, n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
```

```
    for (int i = 0; i < n; i += 2 * k)
      rep(j, 0, k) {
        ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
        a[i + j + k] = ai - z + (z > ai ? mod : 0);
        ai += (ai + z >= mod ? z - mod : z);
      }
}
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty())
    return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s), n = 1
      << B;
  ll inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i, 0, n) out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv
      % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

Polynomial.h

Description: Các phép toán trên chuỗi lũy thừa modulo x^n trên $\mathbb{Z}_{998244353}$. Đa thức được lưu trong vector vl, hệ số có bậc thấp nhất đứng trước. polyMul(a, b): Tính tích hai đa thức (tích chập thông thường). polyInv(a, n): Tìm b sao cho $ab \equiv 1 \pmod{x^n}$, yêu cầu $a[0] \neq 0$. polyLog(a, n): Tính $\ln a \bmod x^n$, yêu cầu $a[0] = 1$. polyExp(a, n): Tính $e^a \bmod x^n$, yêu cầu $a[0] = 0$. Kết quả của các hàm đều có đúng n phần tử (ngoại trừ polyMul).
Usage: `vl f = {1, 1, 1}; vl g = polyInv(f, 8); // g = 1 - x + x^3 - x^4 + ...`
Time: $\mathcal{O}(n \log n)$ cho phép nhân, nghịch đảo và log; exp có hằng số lớn hơn (khoảng gấp 3 lần log). Status: đã kiểm thử trên Library Checker (Inv/Log/Exp of Formal Power Series)

"NumberTheoreticTransform.h" 197b00, 43 lines

```
vl polyMul(const vl &a, const vl &b) { return conv(a, b); }
vl polyInv(const vl &a, int n) {
  vl b = {modpow(a[0], mod - 2)};
  for (int m = 1; m < n; m *= 2) { // b <- b * (2 - a * b) mod
      x^{2m}
    vl ap(a.begin(), a.begin() + min(sz(a), 2 * m));
    vl t = conv(conv(b, b), ap);
    t.resize(2 * m);
    b.resize(2 * m);
    rep(i, 0, 2 * m) b[i] = ((2 * b[i] - t[i]) % mod + mod) %
        mod;
  }
  b.resize(n);
  return b;
}
vl polyDeriv(vl a) {
  rep(i, 1, sz(a)) a[i - 1] = a[i] * i % mod;
  a.pop_back();
  return a;
}
vl polyInteg(vl a) {
  int n = sz(a) + 1;
  vl iv(n + 1, 1); // iv[i] = i^{-1} mod p
  rep(i, 2, n + 1) iv[i] = (mod - (mod / i) * iv[mod % i] % mod
      ) % mod;
  a.insert(a.begin(), 0);
  rep(i, 1, n) a[i] = a[i] * iv[i] % mod;
  return a;
}
vl polyLog(const vl &a, int n) {
  vl r = polyInteg(conv(polyDeriv(a), polyInv(a, n)));
  r.resize(n);
  return r;
}
vl polyExp(const vl &a, int n) {
  vl b = {1};
  for (int m = 1; m < n; m *= 2) { // b <- b * (1 + a - ln b)
      mod x^{2m}
    vl l = polyLog(b, 2 * m), f(2 * m);
    rep(i, 0, 2 * m) f[i] = ((i < sz(a) ? a[i] : 0) - l[i] +
        mod) % mod;
    f[0] = (f[0] + 1) % mod;
    b = conv(b, f);
    b.resize(2 * m);
  }
  b.resize(n);
  return b;
}
```

FastSubsetTransform.h

Description: Biến đổi vector sang một cơ sở cho phép tính tích chập nhanh với công thức: $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, trong đó $\oplus$ là một trong ba phép AND, OR hoặc XOR. Kích thước của vector a phải là lũy thừa của 2.
Time: $\mathcal{O}(N \log N)$

464cf3, 19 lines

```
void FST(vi &a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step)
      rep(j, i, i + step) {
        int &u = a[j], &v = a[j + step];
        tie(u, v) = inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
        pii(u + v, u - v);                   // XOR
      }
  }
  if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
  FST(a, 0);
  FST(b, 0);
  rep(i, 0, sz(a)) a[i] *= b[i];
  FST(a, 1);
  return a;
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Các phép toán số học modulo. Trước tiên cần thiết lập giá trị mod, sau đó có thể sử dụng cấu trúc này.

<cassert>, "euclid.h" ea1ac7, 22 lines

```
const ll mod = 17; // change to something else
struct Mod {
  ll x;
  Mod(ll y) : Mod(y % mod + mod, 0) {}
  Mod(ll y, int) : x(y < mod ? y : y - mod) {}
  Mod operator+(Mod b) { return {x + b.x, 0}; }
  Mod operator-(Mod b) { return {x - b.x + mod, 0}; }
  Mod operator*(Mod b) { return {x * b.x % mod, 0}; }
  Mod operator/(Mod b) { return *this * invert(b); }
  Mod invert(Mod a) {
    ll x, y, g = euclid(a.x, mod, x, y);
    assert(g == 1);
    return x;
  }
  Mod operator^(ll e) {
    if (!e)
      return 1;
    Mod r = *this ^ (e / 2);
    r = r * r;
    return e & 1 ? *this * r : r;
  }
};
```

ModInverse.h

Time: $\mathcal{O}(\log M)$

"euclid.h", <cassert> 6e98db, 18 lines

```
// Case 1: M is prime.
ll modinvPrime(ll a, ll M) {
  ll r = 1, e = M - 2;
  a %= M;
  if (a < 0)
    a += M;
  for (; e; a = a * a % M, e >>= 1)
    if (e & 1)
      r = r * a % M;
  return r;
}
// Case 2: M is arbitrary. Need gcd(a, M) == 1
ll modinv(ll a, ll M) {
  a = (a % M + M) % M;
  ll x, y, g = euclid(a, M, x, y);
  assert(g == 1); // g != 1 => no inverse
  return (x % M + M) % M;
}
```

ModPow.h

b83e45, 8 lines

```
const ll mod = 1000000007; // faster if const

ll modpow(ll b, ll e) {
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;
  return ans;
}
```

ModLog.h

Description: Trả về giá trị $x > 0$ nhỏ nhất thỏa mãn $a^x \equiv b \pmod{m}$, hoặc trả về -1 nếu không tồn tại x như vậy. Có thể sử dụng modLog(a, 1, m) để tính cấp của a modulo m.
Time: $\mathcal{O}(\sqrt{m})$

c040b8, 11 lines

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll)sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m)
    return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i, 2, n + 2) if (A.count(e = e * f % m)) return n * i -
        A[e];
  return -1;
}
```

ModSum.h

Description: Tính tổng của các cấp số cộng sau khi lấy modulo.
modsum(to, c, k, m) $= \sum_{i=0}^{to-1} (ki + c) \% m$. divsum có chức năng tương tự, nhưng sử dụng phép chia lấy phần nguyên. Time: $O(\log m)$, với hằng số khá lớn. Status: Đã kiểm thử với mọi $|k|, |c|, to, m \leq 50$ và trên Kattis: aladin.

5c5bc5, 18 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to - 1) | 1); }
```

```
ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m;
  c %= m;
  if (!k)
    return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m - 1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Tính $a \cdot b \bmod c$ (hoặc $a^b \bmod c$) với $0 \le a, b \le c \le 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ đối với `modmul`, $\mathcal{O}(\log b)$ đối với `modpow`.

bbbd8f, 12 lines

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
}
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1)
      ans = modmul(ans, b, mod);
  return ans;
}
```

ModSqrt.h

Description: Thuật toán Tonelli-Shanks để tìm căn bậc hai modulo. Tìm x thỏa mãn $x^2 \equiv a \pmod p$. Nghiệm còn lại là $-x \pmod p$.
Time: $\mathcal{O}\left(\log^2 p\right)$ trong trường hợp xấu nhất, $\mathcal{O}(\log p)$ với phần lớn giá trị p.

<cassert> b1cb60, 44 lines

```
// Local modpow that takes explicit modulus (ModPow.h's version
    uses global
// const ll mod)
namespace modsqrt_impl {
ll mpow(ll b, ll e, ll m) {
  ll ans = 1;
  b %= m;
  for (; e; b = b * b % m, e /= 2)
    if (e & 1)
      ans = ans * b % m;
  return ans;
}
} // namespace modsqrt_impl

ll modsqrt(ll a, ll p) {
  using modsqrt_impl::mpow;
  a %= p;
  if (a < 0)
    a += p;
  if (a == 0)
    return 0;
  assert(mpow(a, (p - 1) / 2, p) == 1); // else no solution
  if (p % 4 == 3)
    return mpow(a, (p + 1) / 4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (mpow(n, (p - 1) / 2, p) != p - 1)
    ++n;
  ll x = mpow(a, (s + 1) / 2, p);
  ll b = mpow(a, s, p), g = mpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0)
      return x;
    ll gs = mpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

5.2 Primality

FastEratosthenes.h

Description: Sàng nguyên tố, dùng để tìm tất cả các số nguyên tố nhỏ hơn LIM.
Time: LIM=1e9 $\approx$ 1.5s

6b2912, 24 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S + 1);
  pr.reserve(int(LIM / log(LIM) * 1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2)
    if (!sieve[i]) {
      cp.push_back({i, i * i / 2});
      for (int j = i * i; j <= S; j += 2 * i)
        sieve[j] = 1;
    }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i = idx; i < S + L; idx = (i += p))
        block[i - L] = 1;
    rep(i, 0, min(S, R - L)) if (!block[i]) pr.push_back((L + i
        ) * 2 + 1);
  }
  for (int i : pr)
    isPrime[i] = 1;
  return pr;
}
```

LinearSieve.h

Description: Sàng tuyến tính: mỗi hợp số $i \cdot p$ chỉ bị gạch đúng một lần, bởi ước nguyên tố nhỏ nhất của nó. Tính cho mọi $i \le$ LIM: lp[i] = ước nguyên tố nhỏ nhất, phi[i] = hàm Euler, mu[i] = hàm Mobius, và pr = danh sách các số nguyên tố tăng dần. Với LIM $= 10^7$ ba mảng int tốn khoảng 120MB, bỏ bớt mảng không cần dùng.
Hàm Euler: $\phi(n)$ = số nguyên dương không vượt quá n và nguyên tố cùng nhau với n. $\phi(1) = 1$, $\phi(p^k) = (p-1)p^{k-1}$ với p nguyên tố, và nhân tính: $\gcd(m,n) = 1 \Rightarrow \phi(mn) = \phi(m)\phi(n)$. Nếu $n = p_1^{k_1} \cdots p_r^{k_r}$ thì $\phi(n) = n\prod_{p|n}(1 - 1/p)$. $\sum_{d|n} \phi(d) = n$. $\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2$ với $n > 1$. Định lý Euler: $\gcd(a,n) = 1 \Rightarrow a^{\phi(n)} \equiv 1 \pmod n$. Định lý nhỏ Fermat: p nguyên tố, $p \nmid a \Rightarrow a^{p-1} \equiv 1 \pmod p$.
Hàm Mobius: $\mu(n) = 0$ nếu n có thừa số chính phương, ngược lại $\mu(n) = (-1)^k$ với k là số ước nguyên tố phân biệt của n. $\sum_{d|n} \mu(d) = [n = 1]$.

Usage: `linearSieve();` gọi một lần, sau đó dùng lp, phi, mu, pr với $n \le$ LIM. Xem ví dụ ở cuối file. Time: O(LIM) Status: chưa test

873faf, 52 lines

```
const int LIM = 1e6;
int lp[LIM + 1], phi[LIM + 1], mu[LIM + 1];
vi pr;
void linearSieve() {
  phi[1] = mu[1] = 1;
  rep(i, 2, LIM + 1) {
    if (!lp[i])
      lp[i] = i, pr.push_back(i), phi[i] = i - 1, mu[i] = -1;
    for (int p : pr) {
      if (p > lp[i] || (ll)i * p > LIM)
        break;
      lp[i * p] = p;
      if (p == lp[i])
        phi[i * p] = phi[i] * p, mu[i * p] = 0;
      else
        phi[i * p] = phi[i] * (p - 1), mu[i * p] = -mu[i];
    }
  }
}
// Các thừa số nguyên tố của n (có lặp), O(log n)
for (int x = n; x > 1; x /= lp[x])
  cout << lp[x] << ' ';
// Phân tích dạng p^e, và số ước tau(n) = tích (e+1)
int tau = 1;
for (int x = n; x > 1;) {
  int p = lp[x], e = 0;
  while (x % p == 0)
    x /= p, e++;
  tau *= e + 1; // n có thừa số p^e
}
// Kiểm tra nguyên tố: n >= 2 && lp[n] == n
// Tổng tiền tố phi (dùng ll vì tổng vượt int),mu tương tự
vector<ll> sp(LIM + 1);
rep(i, 1, LIM + 1) sp[i] = sp[i - 1] + phi[i];
// phi của một số n lớn (n <= LIM*LIM, ví dụ LIM = 1e6 thì n <=
    1e12):
ll phiOne(ll n) {
  ll r = n;
  for (int p : pr) {
    if ((ll)p * p > n)
      break;
    if (n % p == 0) {
      while (n % p == 0)
        n /= p;
      r -= r / p;
    }
  }
  if (n > 1)
    r -= r / n;
  return r;
}
// Nghịch đảo modulo m khi biết phi(m): a^(-1) = a^(phi(m)-1)
    mod m, với gcd(a,
// m) = 1.
```

PrimeCounting.h

Description: Đếm số nguyên tố $\le n$, tức $\pi(n)$. Chỉ cần quan tâm các giá trị dạng $\lfloor n/i \rfloor$ (có $2\sqrt{n}$ giá trị). Gọi $S(v,p)$ là số $x \in [2,v]$ mà x là nguyên tố hoặc có ước nguyên tố nhỏ nhất $> p$. Khi đó $S(v,p) = S(v,p-1) - (S(v/p,p-1) - S(p-1,p-1))$ nếu p nguyên tố và $p^2 \le v$, ngược lại giữ nguyên. Bắt đầu từ $S(v,1) = v - 1$. lo[i] lưu $S(i,\cdot)$, hi[i] lưu $S(\lfloor n/i \rfloor,\cdot)$. Chỉ cần đổi hàm khởi tạo và phép trừ là đếm được các hàm khác (ví dụ tổng các số nguyên tố, với hàm nhân tính hoàn toàn).
Usage: `ll c = primeCount(1e10);`
Time: $\mathcal{O}\left(n^{3/4}\right)$, bộ nhớ $\mathcal{O}\left(\sqrt{n}\right)$

48703f, 24 lines

```
ll primeCount(ll n) {
  if (n < 2)
```

```
    return 0;
  ll r = sqrtl(n);
  while (r * r > n)
    r--;
  while ((r + 1) * (r + 1) <= n)
    r++;
  vector<ll> lo(r + 1), hi(r + 1);
  rep(i, 1, r + 1) lo[i] = i - 1, hi[i] = n / i - 1;
  for (ll p = 2; p <= r; p++) {
    if (lo[p] == lo[p - 1])
      continue; // p không nguyên tố
    ll pc = lo[p - 1], p2 = p * p;
    ll end = min(r, n / p2);
    for (ll i = 1; i <= end; i++) {
      ll d = i * p;
      hi[i] -= (d <= r ? hi[d] : lo[n / d]) - pc;
    }
    for (ll i = r; i >= p2; i--)
      lo[i] -= lo[i / p] - pc;
  }
  return hi[1];
}
```

MillerRabin.h

Description: Thuật toán Miller-Rabin xác định để kiểm tra tính nguyên tố. Đảm bảo hoạt động chính xác với các số không vượt quá $7 \cdot 10^{18}$; với các số lớn hơn, hãy sử dụng Python và mở rộng tập cơ sở A bằng các giá trị được chọn ngẫu nhiên.

Time: Gấp 7 lần độ phức tạp của phép tính a^b mod c.

"ModMulLL.h" 60dcd1, 14 lines

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1)
    return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
    s = __builtin_ctzll(n - 1), d = n >> s;
  for (ull a : A) { // ^ count trailing zeroes
    ull p = modpow(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n - 1 && i != s)
      return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d8d98d, 18 lines

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

5.3 Divisibility

euclid.h

Description: Tìm hai số nguyên x và y sao cho $ax + by = \gcd(a, b)$. Nếu chỉ cần tính GCD, hãy sử dụng hàm có sẵn __gcd. Nếu a và b nguyên tố cùng nhau thì x chính là nghịch đảo của a modulo b.

33ba8f, 6 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b)
    return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a / b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.

crt(a, m, b, n) computes x such that $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.

Time: $\log(n)$

<cassert>, "euclid.h" 04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

5.3.1 Đẳng thức Bézout

Với $a \neq 0$, $b \neq 0$, đặt $d = \gcd(a, b)$. Khi đó, d là số nguyên dương nhỏ nhất sao cho tồn tại các số nguyên x, y thỏa mãn

$$ax + by = d$$

Nếu (x, y) là một nghiệm thì mọi nghiệm đều có dạng

$$\left(x + \frac{kb}{\gcd(a, b)}, y - \frac{ka}{\gcd(a, b)}\right), \quad k \in \mathbb{Z}$$

5.4 Combinatorics modulo M

Combinatorics.h

Description: Tính giai thừa, giai thừa nghịch đảo và tổ hợp nCr modulo số nguyên tố M. Gọi initFact() một lần trước khi sử dụng nCr.

Time: $\mathcal{O}(MAXN)$ tiền xử lý, $\mathcal{O}(1)$ cho mỗi truy vấn

"ModInverse.h" c5d503, 15 lines

```
const ll mod = 1000000007;
const int MAXN = 1000001;
ll fact[MAXN], invfact[MAXN];
void initFact() {
  fact[0] = 1;
  rep(i, 1, MAXN) fact[i] = fact[i - 1] * i % mod;
  invfact[MAXN - 1] = modinvPrime(fact[MAXN - 1], mod);
  for (int i = MAXN - 1; i > 0; i--)
    invfact[i - 1] = invfact[i] * i % mod;
}
ll nCr(int n, int r) {
  if (r < 0 || r > n)
    return 0;
  return fact[n] * invfact[r] % mod * invfact[n - r] % mod;
}
```

nCrAny.h

Description: Tính nCr modulo một số nguyên dương M bất kỳ. Phân tích M thành các lũy thừa nguyên tố p^k, tính nCr modulo từng p^k (tính n! sau khi loại bỏ các thừa số p, rồi nhân lại với p^e), sau đó kết hợp kết quả bằng CRT. Mỗi p^k phải không vượt quá khoảng 1e7 (do cần tạo bảng kích thước p^k), và M phải có thể phân tích bằng phương pháp thử chia (M tối đa khoảng 1e12). Nếu M là số nguyên tố lớn, hãy sử dụng nCr trong Combinatorics.h.

Time: $\mathcal{O}\left(sumofp^k + log_p(n)\right)$ per query

"CRT.h", "ModInverse.h", "euclid.h" e7ce2d, 52 lines

```
ll powmod(ll b, ll e, ll m) {
  ll r = 1 % m;
  b %= m;
  for (; e; b = b * b % m, e >>= 1)
    if (e & 1)
      r = r * b % m;
  return r;
}
// n! with all factors of p removed, mod pk
ll factNoP(ll n, ll p, ll pk, const vector<ll> &f) {
  ll res = 1;
  while (n) {
    res = res * powmod(f[pk - 1], n / pk, pk) % pk * f[n % pk]
        % pk;
    n /= p;
  }
  return res;
}
ll nCrPk(ll n, ll r, ll p, ll k, ll pk) {
  ll e = 0;
  for (ll t = n; t;)
    t /= p, e += t;
  for (ll t = r; t;)
    t /= p, e -= t;
  for (ll t = n - r; t;)
    t /= p, e -= t;
  if (e >= k)
    return 0;
  vector<ll> f(pk);
  f[0] = 1;
  rep(i, 1, pk) f[i] = i % p ? f[i - 1] * i % pk : f[i - 1];
  ll num = factNoP(n, p, pk, f);
  ll den = factNoP(r, p, pk, f) * factNoP(n - r, p, pk, f) % pk
      ;
  return num * modinv(den, pk) % pk * powmod(p, e, pk) % pk;
}
ll nCrAny(ll n, ll r, ll M) {
  if (r < 0 || r > n)
    return 0;
  ll ans = 0, m = 1, x = M;
  for (ll p = 2; p * p <= x; p++)
    if (x % p == 0) {
      ll k = 0, pk = 1;
      while (x % p == 0)
        x /= p, k++, pk *= p;
      ans = crt(ans, m, nCrPk(n, r, p, k, pk), pk);
      m *= pk;
    }
  if (x > 1) {
    ans = crt(ans, m, nCrPk(n, r, x, 1, x), x);
    m *= x;
  }
  return ans % M;
}
```

5.5 Floor blocks

FloorBlocks.h

Description: Duyệt qua các block cực đại $[l, r]$ mà trên đó $\lfloor n/i \rfloor$ không đổi. Có tối đa $2\sqrt{n}$ block. Dùng cho các tổng dạng $\sum_{i=1}^{n} f(i) \cdot \lfloor n/i \rfloor$, với f có tổng tiền tố tính được nhanh (ví dụ $f(i) = i$). Yêu cầu $n \geq 1$.

Usage: Đây là snippet, đặt bên trong hàm.

```
ll n = ...;
for (ll l = 1, r; l <= n; l = r + 1) {
r = n / (n / l);
// với mọi i trong [l, r]:  n / i == n / l
// ví dụ:  ans += (n / l) * (sum(r) - sum(l - 1));
}
```

Time: $\mathcal{O}\left(\sqrt{n}\right)$

88ed1f, 4 lines

```
for (ll l = 1, r; l <= n; l = r + 1) {
  r = n / (n / l);
  // for every i in [l, r]: n / i == n / l
}
```

5.6 Fractions

ContinuedFractions.h

Description: Cho trước N và một số thực $x \geq 0$, tìm phân số xấp xỉ hữu tỉ p/q gần với x nhất, với $p, q \leq N$. Kết quả thỏa mãn $|p/q - x| \leq 1/qN$.
Với hai phân số hội tụ liên tiếp, ta có: $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. ($p_k/q_k$ luân phiên lớn hơn và nhỏ hơn x.)
Nếu x là số hữu tỉ, y cuối cùng sẽ tiến tới ∞. Nếu x là nghiệm của một đa thức bậc 2, các hệ số a cuối cùng sẽ tuần hoàn.

Time: $\mathcal{O}\left(\log N\right)$

dd6c5e, 24 lines

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX;
  d y = x;
  for (;;) {
    ll lim = min(P ? (N - LP) / P : inf, Q ? (N - LQ) / Q : inf
        ),
      a = (ll)floor(y), b = min(a, lim), NP = b * P + LP, NQ =
          b * Q + LQ;
    if (a > b) {
      // If b > a/2, we have a semi-convergent that gives us a
      // better approximation; if b = a/2, we *may* have one.
      // Return {P, Q} here for a more canonical approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q))
             ? make_pair(NP, NQ)
             : make_pair(P, Q);
    }
    if (abs(y = 1 / (y - (d)a)) > 3 * N) {
      return {NP, NQ};
    }
    LP = P;
    P = NP;
    LQ = Q;
    Q = NQ;
  }
}
```

FracBinarySearch.h

Description: Cho trước hàm f và giới hạn N, tìm phân số nhỏ nhất $p/q \in [0, 1]$ sao cho $f(p/q)$ đúng, với $p, q \leq N$. Bạn có thể cho hàm f ném exception nếu tìm được nghiệm chính xác, khi đó có thể bỏ giới hạn N.

Usage: `fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}`

Time: $\mathcal{O}\left(\log N\right)$

27ab3e, 28 lines

```
struct Frac {
  ll p, q;
};
template <class F> Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Đặt hi = 1/0 để tìm kiếm trên kho
      ảng (0, N]
  if (f(lo))
    return lo;
  assert(f(hi));
  while (A || B) {
    ll adv = 0, step = 1; // Di chuyển hi nếu dir, ngược lại di
        chuyển lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
        adv -= step;
        si = 2;
      }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B;
    B = !!adv;
  }
  return dir ? hi : lo;
}
```

5.7 Bộ ba Pythagoras

Mọi bộ ba Pythagoras đều được sinh duy nhất bởi

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

với $m > n > 0$, $k > 0$, $m \perp n$ (nguyên tố cùng nhau), và một trong hai số m, n chẵn.

5.8 Số nguyên tố

$p = 962592769$ thỏa $2^{21} \mid p - 1$, có thể hữu ích (ví dụ cho NTT). Để băm, dùng 970592641 (31 bit), 31443539979727 (45 bit), 3006703054056749 (52 bit). Có 78498 số nguyên tố nhỏ hơn 1 000 000.

Căn nguyên thủy tồn tại modulo mọi lũy thừa nguyên tố p^a, trừ $p = 2, a > 2$, và có đúng $\phi(\phi(p^a))$ căn nguyên thủy. Với $p = 2, a > 2$, nhóm $\mathbb{Z}_{2^a}^{\times}$ đẳng cấu với $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.9 Ước lượng

$\sum_{d|n} d = O(n \log \log n)$.

Số ước của n nhiều nhất khoảng 100 khi $n < 5 \cdot 10^4$, 500 khi $n < 10^7$, 2000 khi $n < 10^{10}$, 200 000 khi $n < 10^{19}$.

5.10 Hàm Mobius

$$\mu(n) = \begin{cases} 0 & n \text{ có ước chính phương} \\ 1 & n \text{ có số lượng ước nguyên tố chẵn} \\ -1 & n \text{ có số lượng ước nguyên tố lẻ} \end{cases}$$

(Số ước nguyên tố tính theo các ước nguyên tố phân biệt, n không có ước chính phương.)

Công thức nghịch đảo Mobius:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Một số công thức hữu ích khác:

$\sum_{d|n} \mu(d) = [n = 1]$ (rất hay dùng)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Tổ hợp (6)

6.1 Hoán vị

6.1.1 Giai thừa

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

6.1.2 Hoán vị không điểm cố định (số lệch)

Hoán vị của một tập sao cho không phần tử nào nằm ở vị trí ban đầu. Ký hiệu $\lfloor x \rceil$ là làm tròn đến số nguyên gần nhất.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

6.1.3 Bổ đề Burnside

Cho nhóm các phép đối xứng G tác động lên tập X. Số phần tử của X *xét tới đối xứng* bằng

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

trong đó X^g là tập các phần tử bị g giữ cố định ($g.x = x$).

Nếu $f(n)$ đếm số cấu hình có độ dài n, bỏ qua đối xứng quay ($G = \mathbb{Z}_n$):

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Phân hoạch

6.2.1 Hàm phân hoạch số

Số cách viết n thành tổng các số nguyên dương, không tính thứ tự.

$$p(0) = 1,\ p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

6.3 Các dãy số thường gặp

6.3.1 Số Stirling loại 1

Số hoán vị của n phần tử có đúng k chu trình.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$

$$\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\dots(x+n-1)$$

6.3.2 Số Stirling loại 2

Số cách phân hoạch n phần tử phân biệt thành đúng k nhóm không phân biệt thứ tự.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k),\ S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

6.3.3 Số Bell

Tổng số cách phân hoạch n phần tử phân biệt:
$B(n) = \sum_k S(n,k)$.
$B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$

6.3.4 Cây không gốc có gán nhãn

Số cây trên n đỉnh: n^{n-2}
Số cách nối k cây có sẵn (kích thước n_i) thành một cây: $n_1 n_2 \cdots n_k n^{k-2}$
Số cây với bậc các đỉnh là d_i:
$(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.3.5 Số Catalan

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$

- Số đường đi đơn điệu nằm dưới đường chéo trong lưới $n \times n$.
- Số dãy ngoặc đúng gồm n cặp ngoặc.
- Số cây nhị phân có $n+1$ lá, mỗi nút có 0 hoặc 2 con.
- Số cây có thứ tự gồm $n+1$ đỉnh.
- Số cách tam giác hóa đa giác lồi $n+2$ cạnh.
- Số hoán vị của $[n]$ không chứa dãy con tăng độ dài 3.

Graph (7)

7.1 Fundamentals

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max|w_i| < \sim 2^{63}$.
Time: $\mathcal{O}(VE)$ 830a8f, 23 lines

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };

void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });

  int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
  rep(i,0,lim) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    }
  }
  rep(i,0,lim) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  }
}
```

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m, where $m[i][j] =$ `inf` if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j, `inf` if no path, or `-inf` if the path goes through a negative-weight cycle.
Time: $\mathcal{O}(N^3)$ 531245, 12 lines

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>>& m) {
  int n = sz(m);
  rep(i,0,n) m[i][i] = min(m[i][i], 0LL);
  rep(k,0,n) rep(i,0,n) rep(j,0,n)
    if (m[i][k] != inf && m[k][j] != inf) {
      auto newDist = max(m[i][k] + m[k][j], -inf);
      m[i][j] = min(m[i][j], newDist);
    }
  rep(k,0,n) if (m[k][k] < 0) rep(i,0,n) rep(j,0,n)
    if (m[i][k] != inf && m[k][j] != inf) m[i][j] = -inf;
}
```

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.
Time: $\mathcal{O}(|V| + |E|)$ d678d8, 8 lines

```
vi topoSort(const vector<vi>& gr) {
  vi indeg(sz(gr)), q;
  for (auto& li : gr) for (int x : li) indeg[x]++;
  rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
  rep(j,0,sz(q)) for (int x : gr[q[j]])
    if (--indeg[x] == 0) q.push_back(x);
  return q;
}
```

Dijkstra.h

Description: Calculates shortest paths from s in a graph with non-negative edge weights. Returns $\{dist, par\}$ where $dist[i]$ is the shortest distance (or inf if unreachable) and $par[i]$ is the parent on the shortest path.
Usage: `auto [dist, par] = dijkstra(s, adj);`
Time: $\mathcal{O}((V+E)\log V)$ 98c27e, 20 lines

```
const ll inf = 1LL << 62;
pair<vector<ll>, vi> dijkstra(int s, const vector<vector<pair<
    int, ll>>>& adj) {
  int n = sz(adj);
  vector<ll> dist(n, inf);
  vi par(n, -1);
  priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<
      pair<ll, int>>> pq;
  dist[s] = 0;
  pq.push({0, s});
  while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d != dist[u]) continue;
    for (auto [v, w] : adj[u])
      if (dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        par[v] = u;
        pq.push({dist[v], v});
      }
  }
  return {dist, par};
}
```

ShortestCycle.h

Description: Finds the length of the shortest cycle (girth) in an unweighted undirected graph. Returns -1 if the graph is acyclic.
Usage: `int len = shortestCycle(adj);`
Time: $\mathcal{O}(V(V+E))$ e29234, 19 lines

```
int shortestCycle(const vector<vi>& adj) {
  int n = sz(adj), ans = INT_MAX;
  rep(s,0,n) {
    vi dist(n, -1), par(n, -1), q = {s};
    dist[s] = 0;
    rep(qi,0,sz(q)) {
      int u = q[qi];
      for (int v : adj[u]) {
        if (dist[v] == -1) {
          dist[v] = dist[u] + 1;
          par[v] = u;
          q.push_back(v);
        } else if (par[u] != v)
          ans = min(ans, dist[u] + dist[v] + 1);
      }
    }
  }
  return ans == INT_MAX ? -1 : ans;
}
```

7.2 Network flow

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}\left(V^2\sqrt{E}\right)$ 0ae1d4, 48 lines

```
struct PushRelabel {
  struct Edge {
    int dest, back;
    ll f, c;
  };
  vector<vector<Edge>> g;
```

```
  vector<ll> ec;
  vector<Edge*> cur;
  vector<vi> hs; vi H;
  PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n) {}

  void addEdge(int s, int t, ll cap, ll rcap=0) {
    if (s == t) return;
    g[s].push_back({t, sz(g[t]), 0, cap});
    g[t].push_back({s, sz(g[s])-1, 0, rcap});
  }

  void addFlow(Edge& e, ll f) {
    Edge &back = g[e.dest][e.back];
    if (!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
    e.f += f; e.c -= f; ec[e.dest] += f;
    back.f -= f; back.c += f; ec[back.dest] -= f;
  }
  ll calc(int s, int t) {
    int v = sz(g); H[s] = v; ec[t] = 1;
    vi co(2*v); co[0] = v-1;
    rep(i,0,v) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c);

    for (int hi = 0;;) {
      while (hs[hi].empty()) if (!hi--) return -ec[s];
      int u = hs[hi].back(); hs[hi].pop_back();
      while (ec[u] > 0)  // discharge u
        if (cur[u] == g[u].data() + sz(g[u])) {
          H[u] = 1e9;
          for (Edge& e : g[u]) if (e.c && H[u] > H[e.dest]+1)
            H[u] = H[e.dest]+1, cur[u] = &e;
          if (++co[H[u]], !--co[hi] && hi < v)
            rep(i,0,v) if (hi < H[i] && H[i] < v)
              --co[H[i]], H[i] = v + 1;
          hi = H[u];
        } else if (cur[u]->c && H[u] == H[cur[u]->dest]+1)
          addFlow(*cur[u], min(ec[u], cur[u]->c));
        else ++cur[u];
    }
  }
  bool leftOfMinCut(int a) { return H[a] >= sz(g); }
};
```

MinCostMaxFlow.h

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(FE\log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

<cassert>, <bits/extc++.h> 135b73, 77 lines

```
const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
  struct edge {
    int from, to, rev;
    ll cap, cost, flow;
  };
  int N;
  vector<vector<edge>> ed;
  vi seen;
  vector<ll> dist, pi;
  vector<edge*> par;

  MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

  void addEdge(int from, int to, ll cap, ll cost) {
    if (from == to) return;
    ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
    ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
  }

  void path(int s) {
    fill(all(seen), 0);
    fill(all(dist), INF);
    dist[s] = 0; ll di;

    __gnu_pbds::priority_queue<pair<ll, int>> q;
    vector<decltype(q)::point_iterator> its(N);
    q.push({ 0, s });

    while (!q.empty()) {
      s = q.top().second; q.pop();
      seen[s] = 1; di = dist[s] + pi[s];
      for (edge& e : ed[s]) if (!seen[e.to]) {
        ll val = di - pi[e.to] + e.cost;
        if (e.cap - e.flow > 0 && val < dist[e.to]) {
          dist[e.to] = val;
          par[e.to] = &e;
          if (its[e.to] == q.end())
            its[e.to] = q.push({ -dist[e.to], e.to });
          else
            q.modify(its[e.to], { -dist[e.to], e.to });
        }
      }
    }
    rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
  }

  pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
      ll fl = INF;
      for (edge* x = par[t]; x; x = par[x->from])
        fl = min(fl, x->cap - x->flow);

      totflow += fl;
      for (edge* x = par[t]; x; x = par[x->from]) {
        x->flow += fl;
        ed[x->to][x->rev].flow -= fl;
      }
    }
    rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost/2};
  }

  // If some costs can be negative, call this before maxflow:
  void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
      rep(i,0,N) if (pi[i] != INF)
        for (edge& e : ed[i]) if (e.cap)
          if ((v = pi[i] + e.cost) < pi[e.to])
            pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
  }
};
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $O(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.

<cassert> 482fe0, 36 lines

```
template<class T> T edmondsKarp(vector<unordered_map<int, T>>&
    graph, int source, int sink) {
  assert(source != sink);
  T flow = 0;
  vi par(sz(graph)), q = par;

  for (;;) {
    fill(all(par), -1);
    par[source] = 0;
    int ptr = 1;
    q[0] = source;

    rep(i,0,ptr) {
      int x = q[i];
      for (auto e : graph[x]) {
        if (par[e.first] == -1 && e.second > 0) {
          par[e.first] = x;
          q[ptr++] = e.first;
          if (e.first == sink) goto out;
        }
      }
    }
    return flow;
out:
    T inc = numeric_limits<T>::max();
    for (int y = sink; y != source; y = par[y])
      inc = min(inc, graph[par[y]][y]);

    flow += inc;
    for (int y = sink; y != source; y = par[y]) {
      int p = par[y];
      if ((graph[p][y] -= inc) <= 0) graph[p].erase(y);
      graph[y][p] += inc;
    }
  }
}
```

Dinic.h

Description: Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.

d7f0f1, 42 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
```

```
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
};
```

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s, only traversing edges with positive residual capacity.

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time: $\mathcal{O}\left(V^3\right)$

8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i};
  rep(ph,1,n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with prio. queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();
      rep(i,0,n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations

"PushRelabel.h" 0418b3, 13 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vi par(N);
  rep(i,1,N) {
    PushRelabel D(N); // Dinic also works
    for (Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
    tree.push_back({i, par[i], D.calc(i, par[i])});
    rep(j,i+1,N)
      if (par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
  }
  return tree;
}
```

7.3 Matching

HopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Time: $\mathcal{O}\left(E\sqrt{V}\right)$

731cfb, 20 lines

```
int hopcroftKarp(vector<vi>& g, vi& r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  };
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i,0,t) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    }
    if (!f) return res;
    rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
  }
}
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and *btoa* should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vi btoa(m, -1); dfsMatching(g, btoa);`
Time: $\mathcal{O}(VE)$

522b98, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
  if (btoa[j] == -1) return 1;
  vis[j] = 1; int di = btoa[j];
  for (int e : g[di])
    if (!vis[e] && find(e, g, btoa, vis)) {
      btoa[e] = di;
      return 1;
    }
  return 0;
}
int dfsMatching(vector<vi>& g, vi& btoa) {
  vi vis;
  rep(i,0,sz(g)) {
    vis.assign(sz(btoa), 0);
    for (int j : g[i])
      if (find(j, g, btoa, vis)) {
        btoa[j] = i;
        break;
      }
  }
  return sz(btoa) - (int)count(all(btoa), -1);
}
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

<cassert>, "DFSMatching.h" da4196, 20 lines

```
vi cover(vector<vi>& g, int n, int m) {
  vi match(m, -1);
  int res = dfsMatching(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vi q, cover;
  rep(i,0,n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {
      seen[e] = true;
      q.push_back(match[e]);
    }
  }
  rep(i,0,n) if (!lfound[i]) cover.push_back(i);
  rep(i,0,m) if (seen[i]) cover.push_back(n+i);
  assert(sz(cover) == res);
  return cover;
}
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j], and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}\left(N^2M\right)$

8f41c7, 24 lines

```
template<class T>
pair<T, vi> weightedMatching(vector<vector<T>> &C) {
  int i = sz(C), m = i ? sz(C[0]) : 0, c, s, r;
  vector<T> dist(m), pot(m);
  vi match(i), rev(m, -1), cols(m), prev(m);
  T d = 0, nd, cost = 0;
  while (i--) {
    rep(c,0,m) dist[c] = C[i][c], cols[c] = c, prev[c] = i;
    for (s = 0;;) {
      rep(j,s,m) {
        c = cols[j], nd = dist[c] - pot[c];
        if (j == s || d > nd) d = nd, swap(cols[s], cols[j]);
      }
      if ((r = rev[c = cols[s++]]) == -1) break;
      rep(j,0,m) if (dist[j] > (nd = C[r][j]-C[r][c]+dist[c]))
        dist[j] = nd, prev[j] = r;
    }
    cost += dist[c];
    while (s--) pot[cols[s]] = dist[cols[s]] - d;
    for (; r != i; swap(c, match[r]))
      r = rev[c] = prev[c];
  }
  return {cost, match};
}
```

GeneralMatching.h

Description: Matching for general graphs. Fails with probability N/mod.
Time: $\mathcal{O}\left(N^3\right)$

<cassert>, "../numerical/MatrixInverse-mod.h" cb1912, 40 lines

```
vector<pii> generalMatching(int N, vector<pii>& ed) {
  vector<vector<ll>> mat(N, vector<ll>(N)), A;
  for (pii pa : ed) {
    int a = pa.first, b = pa.second, r = rand() % mod;
    mat[a][b] = r, mat[b][a] = (mod - r) % mod;
  }

  int r = matInv(A = mat), M = 2*N - r, fi, fj;
  assert(r % 2 == 0);

  if (M != N) do {
    mat.resize(M, vector<ll>(M));
    rep(i,0,N) {
      mat[i].resize(M);
```

```
      rep(j,N,M) {
        int r = rand() % mod;
        mat[i][j] = r, mat[j][i] = (mod - r) % mod;
      }
    }
  } while (matInv(A = mat) != M);

  vi has(M, 1); vector<pii> ret;
  rep(it,0,M/2) {
    rep(i,0,M) if (has[i])
      rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
        fi = i; fj = j; goto done;
    } assert(0); done:
    if (fj < N) ret.emplace_back(fi, fj);
    has[fi] = has[fj] = 0;
    rep(sw,0,2) {
      ll a = modpow(A[fi][fj], mod-2);
      rep(i,0,M) if (has[i] && A[i][fj]) {
        ll b = A[i][fj] * a % mod;
        rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod;
      }
      swap(fi,fj);
    }
  }
  return ret;
}
```

7.4 DFS algorithms

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: `scc(graph, [&](vi& v) { ... })` visits all components in reverse topological order. `comp[i]` holds the component index of a node (a component only has edges to components with lower index). `ncomps` will contain the number of components.
Time: $\mathcal{O}(E+V)$

76b5c9, 24 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont); cont.clear();
    ncomps++;
  }
  return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.
Usage: `int eid = 0; ed.resize(N);`
`for each edge (a,b) {`
`ed[a].emplace_back(b, eid);`
`ed[b].emplace_back(a, eid++); }`
`bicomps([&](const vi& edgelist) {...});`
Time: $\mathcal{O}(E+V)$

c6b7c7, 32 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
  int me = num[at] = ++Time, top = me;
  for (auto [y, e] : ed[at]) if (e != par) {
    if (num[y]) {
      top = min(top, num[y]);
      if (num[y] < me)
        st.push_back(e);
    } else {
      int si = sz(st);
      int up = dfs(y, e, f);
      top = min(top, up);
      if (up == me) {
        st.push_back(e);
        f(vi(st.begin() + si, st.end()));
        st.resize(si);
      }
      else if (up < me) st.push_back(e);
      else { /* e is a bridge */ }
    }
  }
  return top;
}

template<class F>
void bicomps(F f) {
  num.assign(sz(ed), 0);
  rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (`~x`).
Usage: `TwoSat ts(number of boolean variables);`
`ts.either(0, ~3); // Var 0 is true or var 3 is false`
`ts.setValue(2); // Var 2 is true`
`ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true`
`ts.solve(); // Returns true iff it is solvable`
`ts.values[0..N-1] holds the assigned values to the vars`
Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.

5f9706, 56 lines

```
struct TwoSat {
  int N;
  vector<vi> gr;
  vi values; // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), gr(2*n) {}

  int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x) { either(x, x); }

  void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
  }

  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
  }
};
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V+E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
    if (it == end){ ret.push_back(x); s.pop_back(); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
  return {ret.rbegin(), ret.rend()};
}
```

7.5 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
  vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
  for (pii e : eds) ++cc[e.first], ++cc[e.second];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vector<vi> adj(N, vi(ncols, -1));
  for (pii e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
      int left = fan[i], right = fan[++i], e = cc[i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
      free[right] = e;
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);
  }
  rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
  return ret;
}
```

7.6 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.
Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
  if (!P.any()) { if (!X.any()) f(R); return; }
  auto q = (P | X)._Find_first();
  auto cands = P & ~eds[q];
  rep(i,0,sz(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
  }
}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.
Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

f7c0bc, 49 lines

```
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  }
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        }
        if (j > 0) T[j - 1].d = 0;
        rep(k,mnk,mxk + 1) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
  }
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.

7.7 Trees

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.
Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$

bfce85, 25 lines

```
vector<vi> treeJump(vi& P){
  int on = 1, d = 1;
  while(on < sz(P)) on *= 2, d++;
  vector<vi> jmp(d, P);
  rep(i,1,d) rep(j,0,sz(P))
    jmp[i][j] = jmp[i-1][jmp[i-1][j]];
  return jmp;
}
int jmp(vector<vi>& tbl, int nod, int steps){
  rep(i,0,sz(tbl))
    if(steps&(1<<i)) nod = tbl[i][nod];
  return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b) {
  if (depth[a] < depth[b]) swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int i = sz(tbl); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d) a = c, b = d;
  }
  return tbl[0][a];
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.
Time: $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" 0f62fb, 21 lines

```
struct LCA {
  int T = 0;
  vi time, path, ret;
  RMQ<int> rmq;

  LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C,0,-1), ret)) {}
  void dfs(vector<vi>& C, int v, int par) {
    time[v] = T++;
    for (int y : C[v]) if (y != par) {
      path.push_back(v), ret.push_back(time[v]);
      dfs(C, y, v);
    }
  }

  int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
  //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S|-1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time: $\mathcal{O}(|S| \log |S|)$

<cassert>, "LCA.h" 9775a0, 21 lines

```
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
  static vi rev; rev.resize(sz(lca.time));
  vi li = subset, &T = lca.time;
  auto cmp = [&](int a, int b) { return T[a] < T[b]; };
  sort(all(li), cmp);
  int m = sz(li)-1;
  rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
  }
  sort(all(li), cmp);
  li.erase(unique(all(li)), li.end());
  rep(i,0,sz(li)) rev[li[i]] = i;
  vpi ret = {pii(0, li[0])};
  rep(i,0,sz(li)-1) {
```

```
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  }
  return ret;
}
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.
Time: $\mathcal{O}\left((\log N)^2\right)$

<cassert>, "../data-structures/LazySegmentTree.h" 9547af, 46 lines

```
template <bool VALS_EDGES> struct HLD {
  int N, tim = 0;
  vector<vi> adj;
  vi par, siz, rt, pos;
  Node *tree;
  HLD(vector<vi> adj_)
    : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
      rt(N),pos(N),tree(new Node(0, N)){ dfsSz(0); dfsHld(0); }
  void dfsSz(int v) {
    for (int& u : adj[v]) {
      adj[u].erase(find(all(adj[u]), v));
      par[u] = v;
      dfsSz(u);
      siz[v] += siz[u];
      if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
    }
  }
  void dfsHld(int v) {
    pos[v] = tim++;
    for (int u : adj[v]) {
      rt[u] = (u == adj[v][0] ? rt[v] : u);
      dfsHld(u);
    }
  }
  template <class B> void process(int u, int v, B op) {
    for (;; v = par[rt[v]]) {
      if (pos[u] > pos[v]) swap(u, v);
      if (rt[u] == rt[v]) break;
      op(pos[rt[v]], pos[v] + 1);
    }
    op(pos[u] + VALS_EDGES, pos[v] + 1);
  }
  void modifyPath(int u, int v, int val) {
    process(u, v, [&](int l, int r) { tree->add(l, r, val); });
  }
  int queryPath(int u, int v) { // Modify depending on problem
    int res = -1e9;
    process(u, v, [&](int l, int r) {
        res = max(res, tree->query(l, r));
    });
    return res;
  }
  int querySubtree(int v) { // modifySubtree is similar
    return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
  }
};
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.
Time: All operations take amortized $\mathcal{O}(\log N)$.

<cassert> 0fb462, 90 lines

```
struct Node { // Splay tree. Root's pp contains tree's parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  }
  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  }
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  }
};

struct LinkCut {
  vector<Node> node;
  LinkCut(int N) : node(N) {}

  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  }
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;
      x->fix();
    }
  }
  bool connected(int u, int v) { // are u, v in the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  }
  void makeRoot(Node* u) {
    access(u);
    u->splay();
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    }
  }
  Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; }
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 39e620, 60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
```

```
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--, mat[b][b]++` (and `mat[b][a]--, mat[a][a]++` if G is undirected). Remove the ith row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; }
};
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: `bool left = sideOf(p1,p2,q)==1;`

"Point.h" ca4665, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(P s, P e, P p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

OnSegment.h

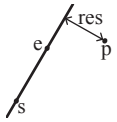
Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.

"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.



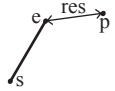
"Point.h" 53630a, 4 lines

```
template<class P>
double lineDist(P a, P b, P p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

Description:
Returns the shortest distance between point p and the line segment from point s to e.

Usage:
```
Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;
```

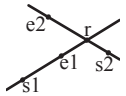


"Point.h" d5f39f, 6 lines

```
typedef Point<double> P;
double segDist(P s, P e, P p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



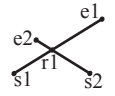
Usage:
```
auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
```

"Point.h" a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
}
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



Usage:
```
vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
```

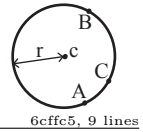
"Point.h", "OnSegment.h" 9d57f2, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

8.2 Circles

circumcircle.h

Description:
The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



"Point.h" 6cffc5, 9 lines

```
typedef Point<double> P;
double ccRadius(P A, P B, P C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(P A, P B, P C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

<cassert>, "Point.h" 84d6d3, 11 lines

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
```

```
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

8.3 Polygons

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  }
  return cnt;
}
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.



Time: $\mathcal{O}(n \log n)$

"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

<cassert>, "Point.h" ac41a6, 17 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  }
  return ret.second;
}
```

Strings (9)

KMP.h

Description: pi[x] tính độ dài của tiền tố dài nhất của s kết thúc tại vị trí x, nhưng không được bằng chính s[0...x] (abacaba -> 0010123). Có thể dùng để tìm tất cả các vị trí xuất hiện của một chuỗi con. Time: O(n) Status: Đã kiểm chứng trên kattis:stringmatching

d4375c, 16 lines

```
vi pi(const string &s) {
  vi p(sz(s));
  rep(i, 1, sz(s)) {
    int g = p[i - 1];
    while (g && s[i] != s[g])
      g = p[g - 1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}
vi match(const string &s, const string &pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i, sz(p) - sz(s), sz(p)) if (p[i] == sz(pat))
    res.push_back(i - 2 * sz(pat));
  return res;
}
```

Zfunc.h

Description: z[i] tính độ dài tiền tố chung dài nhất của hai chuỗi s[i:] và s, ngoại trừ z[0] = 0. Ví dụ: abacaba -> 0010301.

Time: $\mathcal{O}(n)$

ee09e2, 12 lines

```
vi Z(const string &S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i, 1, sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

Manacher.h

Description: Với mỗi vị trí trong chuỗi, tính: p[0][i] = bán kính palindrome chẵn dài nhất có tâm tại vị trí i. p[1][i] = bán kính palindrome lẻ dài nhất có tâm tại vị trí i (không tính ký tự ở chính giữa).

Time: $\mathcal{O}(N)$

e7ad79, 15 lines

```
array<vi, 2> manacher(const string &s) {
  int n = sz(s);
  array<vi, 2> p = {vi(n + 1), vi(n)};
  rep(z, 0, 2) for (int i = 0, l = 0, r = 0; i < n; i++) {
    int t = r - i + !z;
    if (i < r)
      p[z][i] = min(t, p[z][l + t]);
    int L = i - p[z][i], R = i + p[z][i] - !z;
    while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
      p[z][i]++, L--, R++;
    if (R > r)
      l = L, r = R;
  }
  return p;
}
```

MinRotation.h

Description: Tìm phép xoay vòng có thứ tự từ điển nhỏ nhất của một chuỗi.

Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`

Time: $\mathcal{O}(N)$

d07a42, 15 lines

```
int minRotation(string s) {
  int a = 0, N = sz(s);
  s += s;
  rep(b, 0, N) rep(k, 0, N) {
    if (a + k == b || s[a + k] < s[b + k]) {
      b += max(0, k - 1);
      break;
    }
    if (s[a + k] > s[b + k]) {
      a = b;
      break;
    }
  }
  return a;
}
```

Hashing.h

Description: Các hàm hỗ trợ băm chuỗi (String Hashing).

2d2a67, 52 lines

```
// Phép toán modulo 2^64−1. Chậm hơn khoảng 2 lần so với modulo 2^64
// và cần nhiều code hơn, nhưng hoạt động tốt trên các bộ test có chủ đích
// (ví dụ: chuỗi Thue–Morse, trong đó ABBA... và BAAB... có độ dài 2^10 cho cùng
// hash nếu sử dụng modulo 2^64). Có thể thay bằng "typedef ull H;" nếu cho rằng
// dữ liệu test là ngẫu nhiên, hoặc sử dụng modulo 10^9+7 nếu Birthday Paradox
// không phải vấn đề.
typedef uint64_t ull;
struct H {
  ull x;
  H(ull x = 0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) {
    auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64);
  }
  ull get() const { return x + !~x; }
```

```
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11 + 3; // (order ~ 3e9; random also
    ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string &str) : ha(sz(str) + 1), pw(ha) {
    pw[0] = 1;
    rep(i, 0, sz(str)) ha[i + 1] = ha[i] * C + str[i],
                  pw[i + 1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};

vector<H> getHashes(string &str, int length) {
  if (sz(str) < length)
    return {};
  H h = 0, pw = 1;
  rep(i, 0, length) h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  rep(i, length, sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i - length]);
  }
  return ret;
}

H hashString(string &s) {
  H h{};
  for (char c : s)
    h = h * C + c;
  return h;
}
```

AhoCorasick.h

Description: Dùng để tìm kiếm nhiều mẫu (pattern) cùng lúc. Khởi tạo bằng AhoCorasick ac(patterns); trạng thái gốc của automaton có chỉ số 0. find(word) trả về tại mỗi vị trí ID của từ dài nhất kết thúc tại đó, hoặc -1 nếu không có từ nào kết thúc tại vị trí đó. findAll(-, word) tìm tất cả các từ (tối đa O(N * sqrt(N)) từ nếu không có pattern trùng nhau), nhóm theo vị trí bắt đầu và sắp xếp từ ngắn đến dài. Cho phép các pattern trùng nhau; không cho phép pattern rỗng. Muốn tìm từ dài nhất bắt đầu tại mỗi vị trí, hãy đảo ngược tất cả đầu vào. Với bảng chữ cái lớn, chia mỗi ký tự thành các nhóm bit và dùng bit đánh dấu để phân biệt ranh giới giữa các ký tự. Time: - Xây dựng: O(26N), với N là tổng độ dài các pattern. - find(x): O(N), với N là độ dài x. - findAll: O(NM).

<cassert> f35677, 71 lines

```
struct AhoCorasick {
  enum { alpha = 26, first = 'A' }; // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string &s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int &m = N[n].next[c - first];
      if (m == -1) {
        n = m = sz(N);
        N.emplace_back(-1);
      } else
        n = m;
    }
    if (N[n].end == -1)
      N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string> &pat) : N(1, -1) {
    rep(i, 0, sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i, 0, alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1)
          ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start]) =
            N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string> &pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
Time: $\mathcal{O}(\log N)$

edce47, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}

void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add `|| R.empty()`. Returns empty set on failure (or if G is empty).
Time: $\mathcal{O}(N \log N)$

9e9d8d, 19 lines

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
  vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
  T cur = G.first;
  int at = 0;
  while (cur < G.second) { // (A)
    pair<T, int> mx = make_pair(cur, -1);
    while (at < sz(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.
Usage: `constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});`
Time: $\mathcal{O}\left(k \log \frac{n}{k}\right)$

753a4c, 19 lines

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
  if (p == q) return;
  if (from == to) {
    g(i, to, p);
    i = to; p = q;
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid+1, to, f, g, i, p, q);
  }
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
  if (to <= from) return;
```

```
  int i = from; auto p = f(i), q = f(to-1);
  rec(from, to-1, f, g, i, p, q);
  g(i, to, q);
}
```

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the < marked with (A) to <=, and reverse the loop at (B). To minimize f, change it to >, also at (B).
Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`
Time: $\mathcal{O}(\log(b-a))$

<cassert> 9155b4, 11 lines

```
template<class F>
int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  }
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

LIS.h

Description: Compute indices for the longest increasing subsequence.
Time: $\mathcal{O}(N \log N)$

2932a0, 17 lines

```
template<class I> vi lis(const vector<I>& S) {
  if (S.empty()) return {};
  vi prev(sz(S));
  typedef pair<I, int> p;
  vector<p> res;
  rep(i,0,sz(S)) {
    // change 0 -> i for longest non-decreasing subsequence
    auto it = lower_bound(all(res), p{S[i], 0});
    if (it == res.end()) res.emplace_back(), it = res.end()-1;
    *it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it-1)->second;
  }
  int L = sz(res), cur = res.back().second;
  vi ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$

b20ccc, 16 lines

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
  while (b < sz(w) && a + w[b] <= t) a += w[b++];
  if (b == sz(w)) return a;
  int m = *max_element(all(w));
  vi u, v(2*m, -1);
  v[a+m-t] = b;
  rep(i,b,sz(w)) {
    u = v;
    rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
    for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
      v[x-w[j]] = max(v[x-w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
}
```

10.3 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \leq f(a,d)$ and $f(a,c) + f(b,d) \leq f(a,d) + f(b,c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.
Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i)\leq k<hi(i)}(f(i,k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R-1$.
Time: $\mathcal{O}((N + (hi - lo))\log N)$

d38d2b, 18 lines

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
  }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

DigitDP.h

Description: Digit DP template to count numbers in range [0, N] satisfying certain conditions. For range [L, R], compute solve(R) - solve(L - 1).
Time: $\mathcal{O}(D * S * 10)$. D is number of digits, S is state size.

cb5eda, 38 lines

```
struct DigitDP {
  int memo[20][200]; // Change base on problem
  vi dig;
  int dfs(int pos, int st, bool tight, bool lz) {
    if (pos == sz(dig)) {
      return (st == 0); // Base case
    }
    if (!tight && !lz && memo[pos][st] != -1) {
      return memo[pos][st];
    }
    int lim = tight ? dig[pos] : 9;
    int res = 0;
    for (int d = 0; d <= lim; ++d) {
      bool nlz = lz && (d == 0);
      int nst = transition(st, d, nlz); // Change status
          function
      res += dfs(pos + 1, nst, tight && (d == lim), nlz);
    }
    if (!tight && !lz) {
      memo[pos][st] = res;
    }
    return res;
  }
  int transition(int st, int d, bool nlz) {
    // Customize the state transition formula according to
        the problem.
    // For example: calculate the digit sum, count
        occurrences of a specific digit, etc.
    if (nlz) return st; // If still processing leading zeros,
        keep the state unchanged.
    return st + d;
  }
  int solve(long long n) {
    if (n < 0) return 0;
    if (n == 0) return 1; // Adjust depending on whether zero
        should be counted as valid.
    dig.clear();
    for (; n > 0; n /= 10) dig.push_back(n % 10);
    reverse(all(dig));
    memset(memo, -1, sizeof(memo));
    return dfs(0, 0, true, true);
  }
};
```

10.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- `x & -x` is the least bit in x.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after x with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)];` computes all sums of subsets.

10.5.2 Pragmas

- `#pragma GCC optimize ("Ofast")` will make GCC auto-vectorize loops and optimizes floating points better.
- `#pragma GCC target ("avx2")` can double performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC optimize ("trapv")` kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to a (mod b) in the range $[0, 2b)$.

751a02, 8 lines

```
typedef unsigned long long ull;
struct FastMod {
  ull b, m;
  FastMod(ull b) : b(b), m(-1ULL / b) {}
  ull reduce(ull a) { // a % b + (0 or b)
    return a - (ull)((__uint128_t(m) * a) >> 64) * b;
  }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: `./a.out < input.txt`
Time: About 5x as fast as cin/scanf.

7b3c70, 17 lines

```
inline char gc() { // like getchar()
  static char buf[1 << 16];
  static size_t bc, be;
  if (bc >= be) {
    buf[0] = 0, bc = 0;
    be = fread(buf, 1, sizeof(buf), stdin);
  }
  return buf[bc++]; // returns 0 on EOF
}

int readInt() {
  int a, c;
  while ((a = gc()) < 40);
  if (a == '-') return -readInt();
  while ((c = gc()) >= 48) a = a * 10 + c - 480;
  return a - 48;
}
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

<cassert> 745db2, 8 lines

```
// Either globally or in a single class:
static char buf[450 << 20];
void* operator new(size_t s) {
  static size_t i = sizeof buf;
  assert(s < i);
  return (void*)&buf[i -= s];
}
void operator delete(void*) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

"BumpAllocator.h" 2dd6c9, 10 lines

```
template<class T> struct ptr {
  unsigned ind;
  ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
    assert(ind < sizeof buf);
  }
  T& operator*() const { return *(T*)(buf + ind); }
  T* operator->() const { return &**this; }
  T& operator[](int a) const { return (&**this)[a]; }
  explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.
Usage: `vector<vector<int, small<int>>> ed(N);`

bb66d4, 14 lines

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
  typedef T value_type;
  small() {}
  template<class U> small(const U&) {}
  T* allocate(size_t n) {
    buf_ind -= n * sizeof(T);
    buf_ind &= 0 - alignof(T);
    return (T*)(buf + buf_ind);
  }
  void deallocate(T*, size_t) {}
};
```

SIMD.h

Description: Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern `"_mm(256)?_name_(si(128|256)|epi(8|16|32|64)|pd|ps)"`. Not all are described here; grep for _mm_ in `/usr/lib/gcc/*/4.9/include/` for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and #define __SSE__ and __MMX__ before including it. For aligned memory use `_mm_malloc(size, 32)` or `int buf[N] alignas(32)`, but prefer loadu/storeu.

c9ac08, 43 lines

```
#pragma GCC target ("avx2") // or sse4.1
#include "immintrin.h"

typedef __m256i mi;
#define L(x) _mm256_loadu_si256((mi*)&(x))

// High-level/specific methods:
// load(u)?_si256, store(u)?_si256, setzero_si256, _mm_malloc
// blendv_(epi8|ps|pd) (z?y:x), movemask_epi8 (hibits of bytes)
// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x
// sad_epu8: sum of absolute differences of u8, outputs 4xi64
// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15
// madd_epi16: dot product of signed i16's, outputs 8xi32
// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane
// shuffle_epi8(x, y) takes a vector instead of an imm

// Methods that work with most data types (append e.g. _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|hi)

int sumi32(mi m) { union {int v[8]; mi m;} u; u.m = m;
  int ret = 0; rep(i,0,8) ret += u.v[i]; return ret; }
mi zero() { return _mm256_setzero_si256(); }
mi one() { return _mm256_set1_epi32(-1); }
bool all_zero(mi m) { return _mm256_testz_si256(m, m); }
bool all_one(mi m) { return _mm256_testc_si256(m, one()); }

ll example_filteredDotProduct(int n, short* a, short* b) {
  int i = 0; ll r = 0;
  mi zero = _mm256_setzero_si256(), acc = zero;
  while (i + 16 <= n) {
    mi va = L(a[i]), vb = L(b[i]); i += 16;
    va = _mm256_and_si256(_mm256_cmpgt_epi16(vb, va), va);
    mi vp = _mm256_madd_epi16(va, vb);
    acc = _mm256_add_epi64(_mm256_unpacklo_epi32(vp, zero),
      _mm256_add_epi64(acc, _mm256_unpackhi_epi32(vp, zero)));
  }
  union {ll v[4]; mi m;} u; u.m = acc; rep(i,0,4) r += u.v[i];
  for (;i<n;++i) if (a[i] < b[i]) r += a[i]*b[i]; // <- equiv
  return r;
}
```

Techniques (A)

techniques.txt 159 lines

```
Recursion
Divide and conquer
  Finding interesting points in N log N
Algorithm analysis
  Master theorem
  Amortized time complexity
Greedy algorithm
  Scheduling
  Max contiguous subvector sum
  Invariants
  Huffman encoding
Graph theory
  Dynamic graphs (extra book-keeping)
  Breadth first search
  Depth first search
  * Normal trees / DFS trees
  Dijkstra's algorithm
  MST: Prim's algorithm
  Bellman-Ford
  Konig's theorem and vertex cover
  Min-cost max flow
  Lovasz toggle
  Matrix tree theorem
  Maximal matching, general graphs
  Hopcroft-Karp
  Hall's marriage theorem
  Graphical sequences
  Floyd-Warshall
  Euler cycles
  Flow networks
  * Augmenting paths
  * Edmonds-Karp
  Bipartite matching
  Min. path cover
  Topological sorting
  Strongly connected components
  2-SAT
  Cut vertices, cut-edges and biconnected components
  Edge coloring
  * Trees
  Vertex coloring
  * Bipartite graphs (=> trees)
  * 3^n (special case of set cover)
  Diameter and centroid
  K'th shortest path
  Shortest cycle
Dynamic programming
  Knapsack
  Coin change
  Longest common subsequence
  Longest increasing subsequence
  Number of paths in a dag
  Shortest path in a dag
  Dynprog over intervals
  Dynprog over subsets
  Dynprog over probabilities
  Dynprog over trees
  3^n set cover
  Divide and conquer
  Knuth optimization
  Convex hull optimizations
  RMQ (sparse table a.k.a 2^k-jumps)
  Bitonic cycle
  Log partitioning (loop over most restricted)
Combinatorics
  Computation of binomial coefficients
  Pigeon-hole principle
  Inclusion/exclusion
  Catalan number
  Pick's theorem
Number theory
  Integer parts
  Divisibility
  Euclidean algorithm
  Modular arithmetic
  * Modular multiplication
  * Modular inverses
  * Modular exponentiation by squaring
  Chinese remainder theorem
  Fermat's little theorem
  Euler's theorem
  Phi function
  Frobenius number
  Quadratic reciprocity
  Pollard-Rho
  Miller-Rabin
  Hensel lifting
  Vieta root jumping
Game theory
  Combinatorial games
  Game trees
  Mini-max
  Nim
  Games on graphs
  Games on graphs with loops
  Grundy numbers
  Bipartite games without repetition
  General games without repetition
  Alpha-beta pruning
Probability theory
Optimization
  Binary search
  Ternary search
  Unimodality and convex functions
  Binary search on derivative
Numerical methods
  Numeric integration
  Newton's method
  Root-finding with binary/ternary search
  Golden section search
Matrices
  Gaussian elimination
  Exponentiation by squaring
Sorting
  Radix sort
Geometry
  Coordinates and vectors
  * Cross product
  * Scalar product
  Convex hull
  Polygon cut
  Closest pair
  Coordinate-compression
  Quadtrees
  KD-trees
  All segment-segment intersection
Sweeping
  Discretization (convert to events and sweep)
  Angle sweeping
  Line sweeping
  Discrete second derivatives
Strings
  Longest common substring
  Palindrome subsequences
  Knuth-Morris-Pratt
  Tries
  Rolling polynomial hashes
  Suffix array
  Suffix tree
  Aho-Corasick
  Manacher's algorithm
  Letter position lists
Combinatorial search
  Meet in the middle
  Brute-force with pruning
  Best-first (A*)
  Bidirectional search
  Iterative deepening DFS / A*
Data structures
  LCA (2^k-jumps in trees in general)
  Pull/push-technique on trees
  Heavy-light decomposition
  Centroid decomposition
  Lazy propagation
  Self-balancing trees
  Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
  Monotone queues / monotone stacks / sliding queues
  Sliding queue using 2 stacks
  Persistent segment tree
```